Module 3
Solving Problems by Searching

Nirmala Shinde Baloorkar

Assistant Professor

Department of Computer Engineering

SOMAIYA VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering

Somaiya TRUST

Outline

- Problem Solving Agent
- Well-defined problems and solutions
- Formulating Problems
- Examples
- Searching
- Measuring Problem Solving Performance

Solving Problems By Searching

- Reflex agent base their actions on
  - a direct mapping from states to actions
  - but can’t work well in environments for
    - which this mapping would be too large to store
    - and would take too long to learn
- Hence, goal-based agent is used

Problem-Solving Agent

- Problem-solving agent
  - A kind of goal-based agent
  - It solves problem by
    - Finding sequences of actions that lead to desirable states(goals)
- To solve a problem
  - The first step is the **goal formulation**

Goal Formulation

- Goal formulation : is based on the current situation and the agent's performance measure.
- The goal is formulated
  - As a set of world states, in which the goal is satisfied
- Reaching from initial state -> goal state
  - Actions are required
- Actions are the operators
  - Causing transitions between world states
  - Actions should be abstract enough at a certain degree, instead of very detailed
  - E.g. turn left vs turn left 30^0 etc.

Problem Formulation

- The process of deciding
  - What actions and states to consider, given a goal
- E.g. driving Arad -> Bucharest
  - In-between states and actions defined
  - States : Some places/cities in Arad & Bucharest
  - Actions : Turn left, Turn right, go straight, brake & etc.

Search

- Because there are many ways to achieve the same goal
  - Those ways are together expressed as a tree
  - Multiple options of unknown value at a point,
    - The agent can examine different possible sequence of actions, and choose the best
  - This process of looking for the best sequence is called **search**
  - The best sequence is then a list of actions, called **solution**

Search Algorithm

- Defined as
  - Taking a problem
  - And returns a solution
- Once a solution is found
  - The agent follows the solution
  - And carries out the list of actions – execution phase
- Design of an agent

Formulate-Search-Execute

Problem-Solving Agents

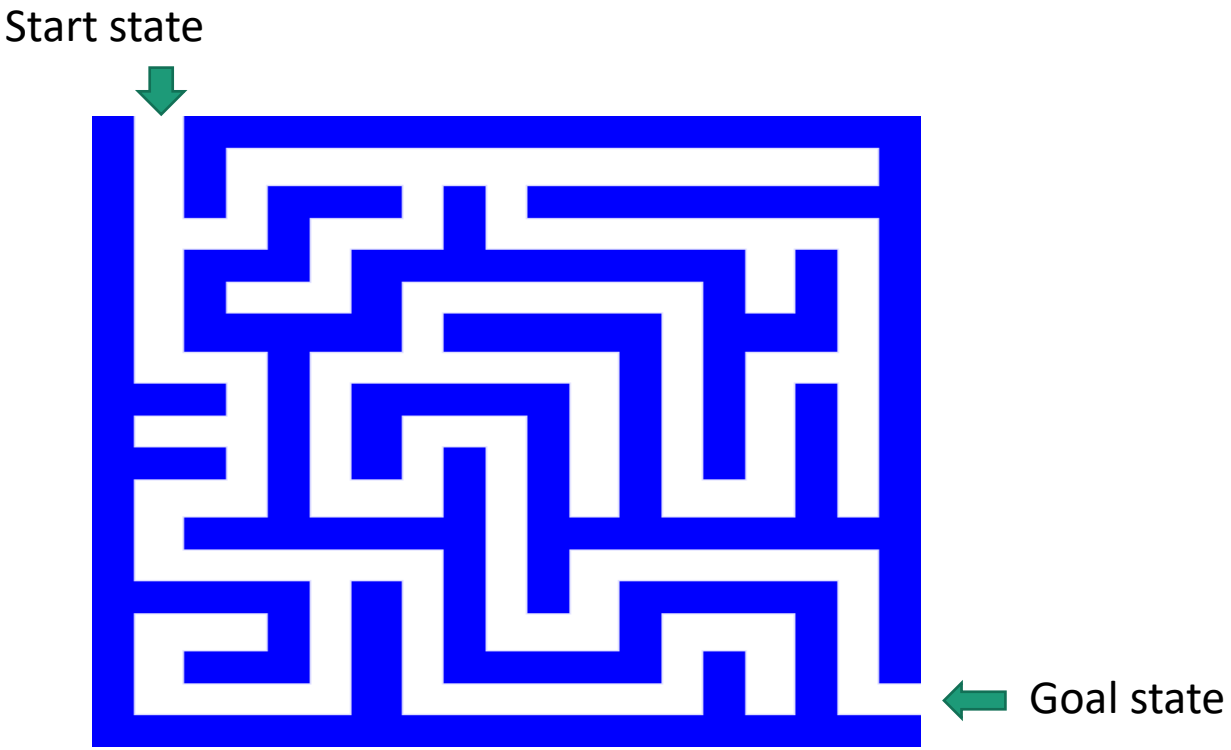
```
function SIMPLE-PROBLEM-SOLVING-AGENT( percept) returns an action
    static: seq, an action sequence, initially empty
            state, some description of the current world state
            goal, a goal, initially null
            problem, a problem formulation

    state ← UPDATE-STATE(state, percept)
    if seq is empty then do
        goal ← FORMULATE-GOAL(state)
        problem ← FORMULATE-PROBLEM(state, goal)
        seq ← SEARCH( problem)
    action ← FIRST(seq)
    seq ← REST(seq)
    return action
```

Desired Environments

- **Static:** because formulating and solving the problem is done without paying attention to any changes that might be occurring in the environment.
- Assumes that the initial state is known; knowing it is easiest if the environment is **observable**
- The idea of enumerating "alternative courses of action“ assumes that the environment can be viewed as **discrete**.
- Finally, and most importantly, the agent design assumes that the environment is **deterministic**

Search

- We will consider the problem of designing **goal-based agents** in **fully observable**, **deterministic**, **discrete**, **known** environments



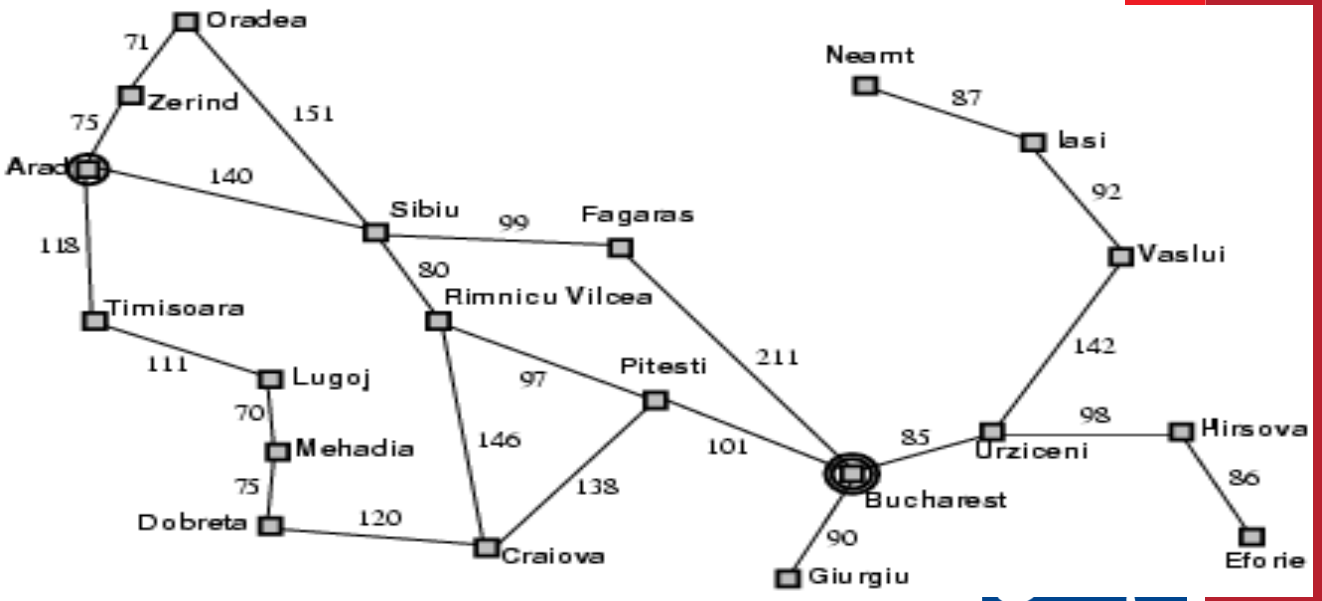
Search

- We will consider the problem of designing **goal-based agents** in **fully observable**, **deterministic**, **discrete**, **known** environments
  - The agent must find a *sequence of actions* that reaches the goal
  - The **performance measure** is defined by (a) reaching the goal and (b) how "expensive" the path to the goal is
  - We are focused on the process of finding the solution

Example: Romania

- On holiday in Romania; currently in Arad
- Flight leaves tomorrow from Bucharest

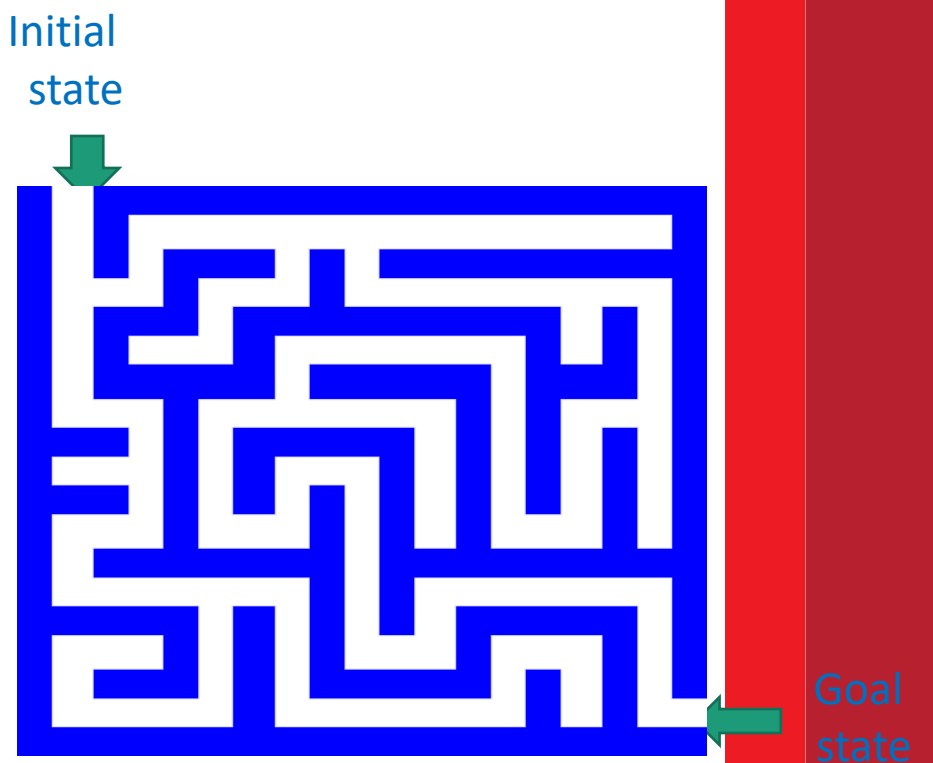
- **Formulate Goal**
  - Be in Bucharest
- **Formulate Problem**
  - States
    - Various cities
  - Actions
    - Drive between cities
- **Find Solution**
  - Sequence of cities
    - E.g. Arad, Sibiu, Fagars etc.



Well-defined Problems & Solutions

A problem can be defined formally by five components

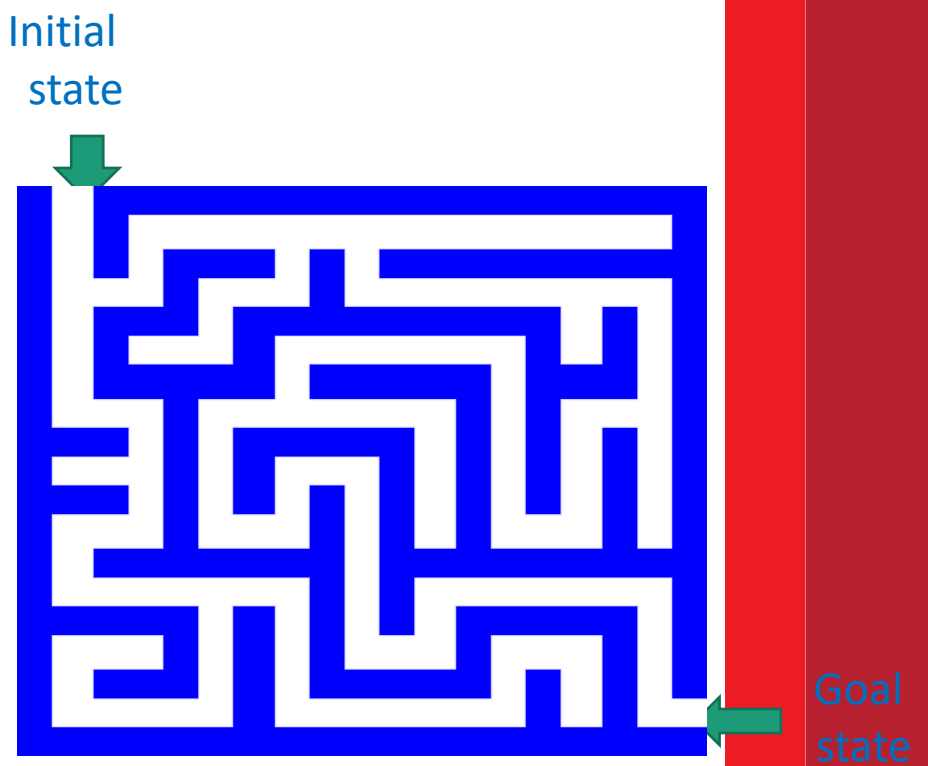
- **Initial state**
  - The agents starts in
- **Actions**
  - set of possible action
- **Transition model**
  - Description of what each action does?
  - **Successor function** refer to any state reachable from given state by a single action



Well-defined Problems & Solutions

- **Goal state/test**
  - Determines whether a given state is goal
- **Path cost**
  - Assigns a numeric cost to each path
  - The a sum of *step costs* of individual actions
  - Step costs are nonnegative

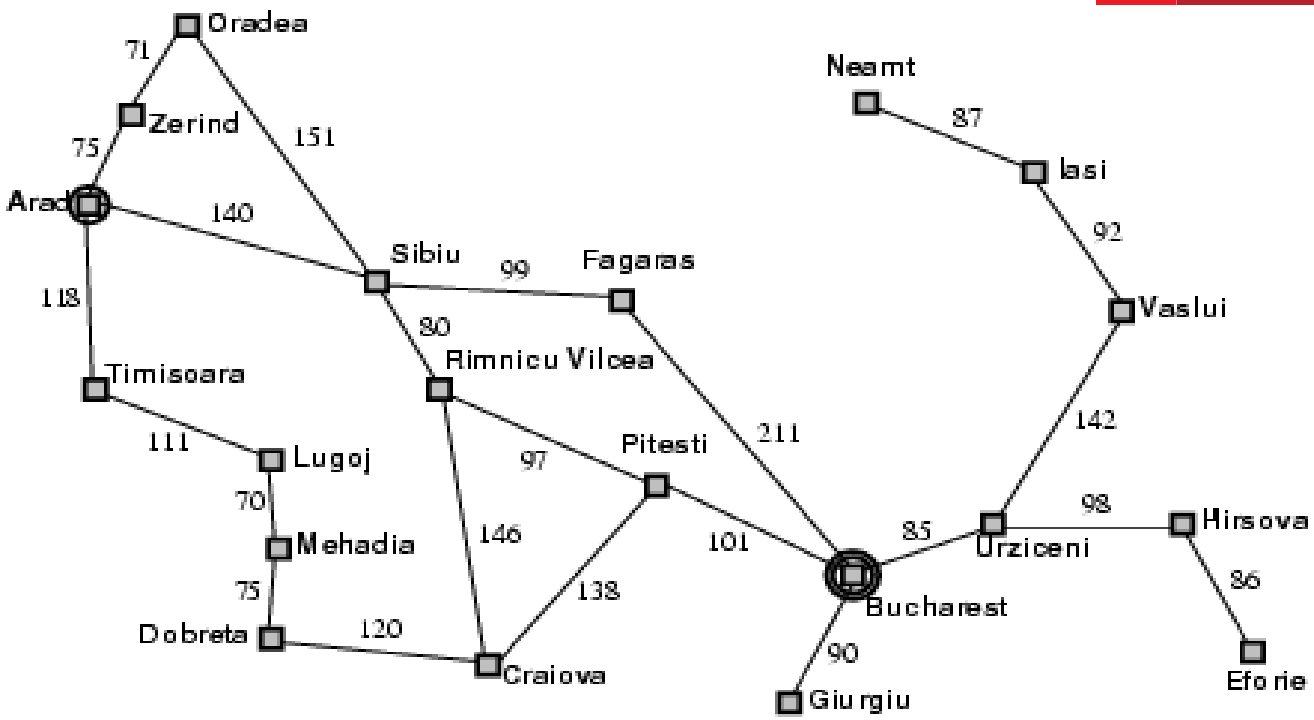
- The **optimal solution** is the sequence of actions that gives the *lowest* path cost for reaching the goal



Example: Romania

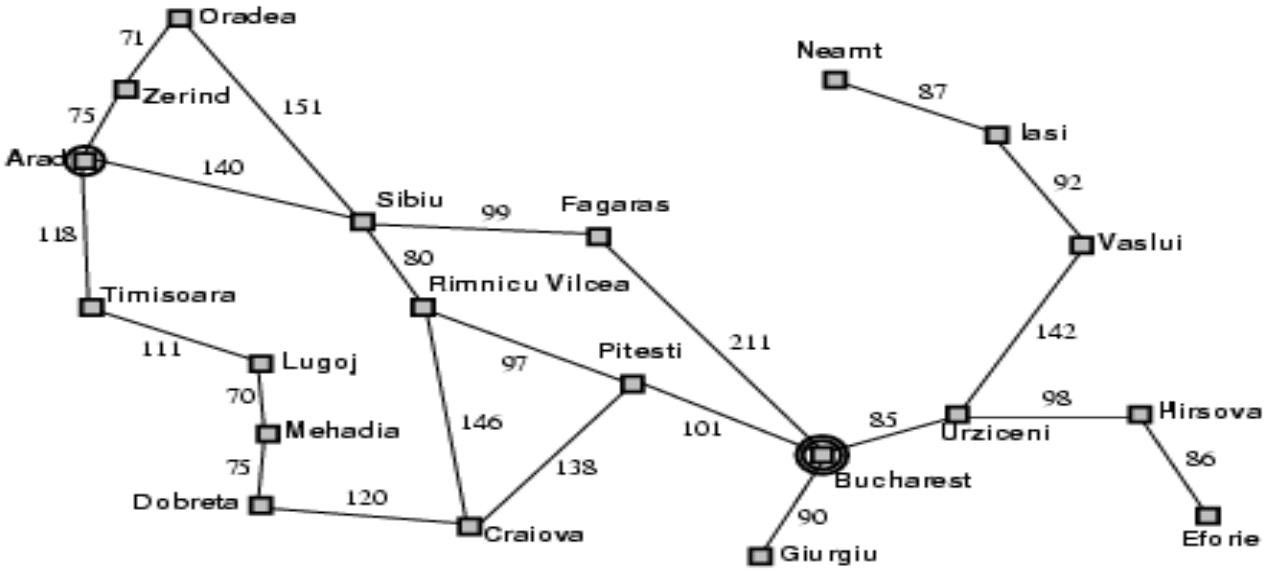
- On holiday in Romania; currently in Arad
- Flight leaves tomorrow from Bucharest

- **Initial state**
  - In (Arad)
- **Actions**
  - Go from one city to another
  - E.g. Go(Sibiu), Go(Timisoara)
- **Transition model**
  - If you go from city A to city B, you end up in city B
  - E.g. In(Arad), Go(Sibiu) = In(Sibiu)
- **Goal state**
  - Be in Bucharest
  - E.g. In(Buchares)
- **Path cost**
  - Sum of edge costs (total distance traveled)



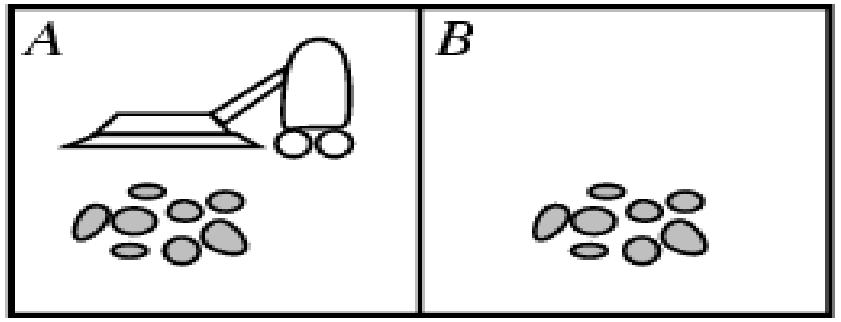
State space

- The initial state, actions, and transition model define the **state space** of the problem
  - The set of all states reachable from initial state by any sequence of actions
  - Can be represented as a **directed graph** where the nodes are states and links between nodes are actions
- What is the state space for the Romania problem?



SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya College of Engineering
Somaiya TRUST

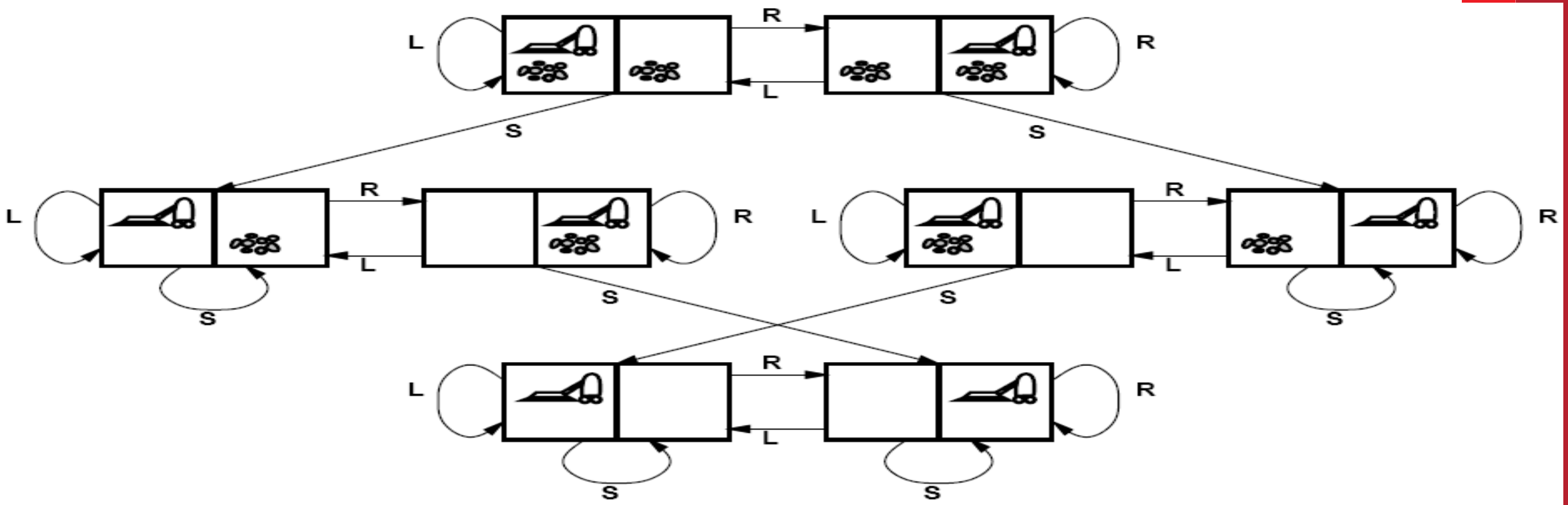
Example: Vacuum world



- **States**
  - Agent location and dirt location
  - How many possible states?
    - $2 \times 2^2 = 8$ possible world states
  - What if there are *n* possible locations?
    - The size of the state space grows exponentially with the "size" of the world! ($n \times 2^n$ world states)
- **Initial State**
  - Any state can be designed as the initial state.
  - E.g. in(A)

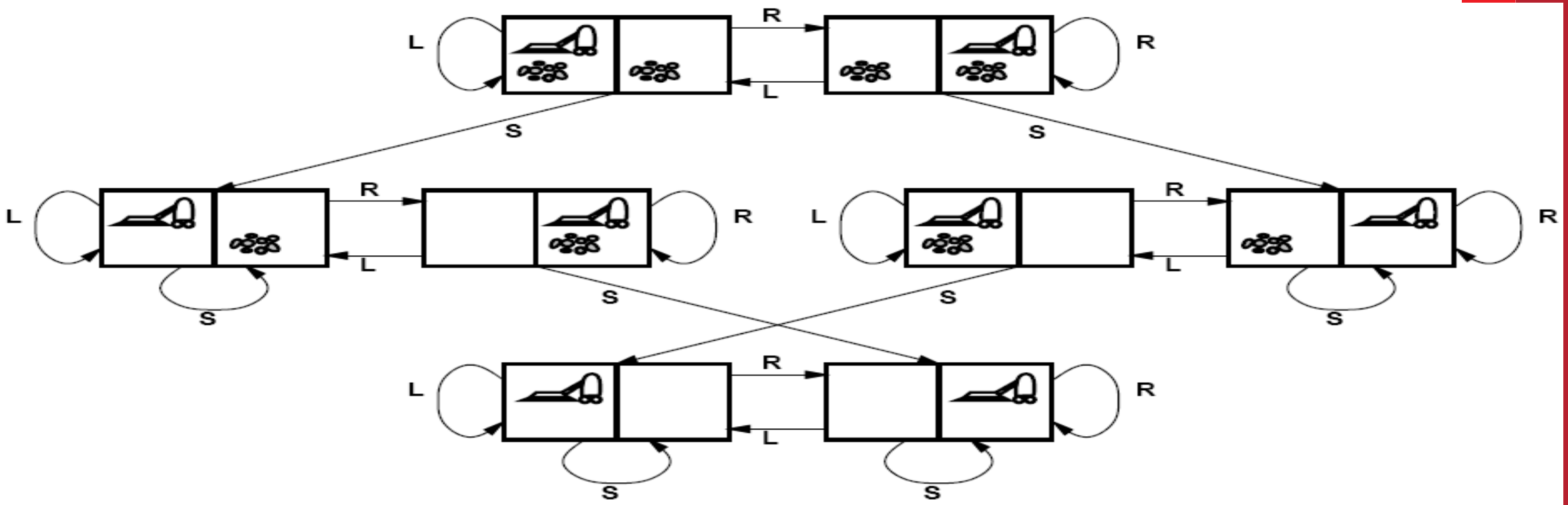
Vacuum world state space graph

- **Actions**
  - Left(L), Right(R), Suck(S)
  - Larger Environment might have Up & Down
- **Transition Model/State Space**
  - Description of what each action does?



Vacuum world state space graph

- **Goal Test**
  - This checks whether all the squares are clean
- **Path Cost**
  - Each Step cost =1
  - The path cost is the number of steps in the path.



Example: The 8-puzzle

- **States**
  - Locations of tiles
    - 8-puzzle: 181,440 states (9!/2)
    - 15-puzzle: ~10 trillion states
    - 24-puzzle: ~10^{25} states
- **Actions/Operators**
  - Move blank left, right, up, down
- **Goal Test**
  - The current state matches a goal state
- **Path cost**
  - 1 per move

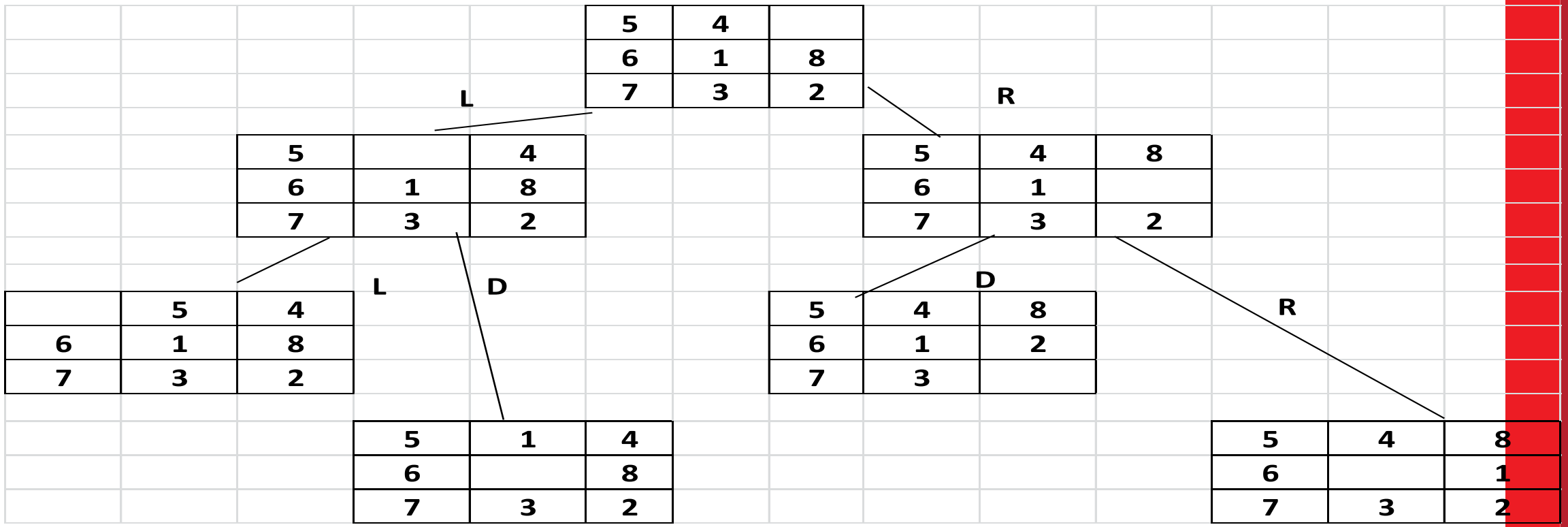
5	4	
6	1	8
7	3	2

Initial State

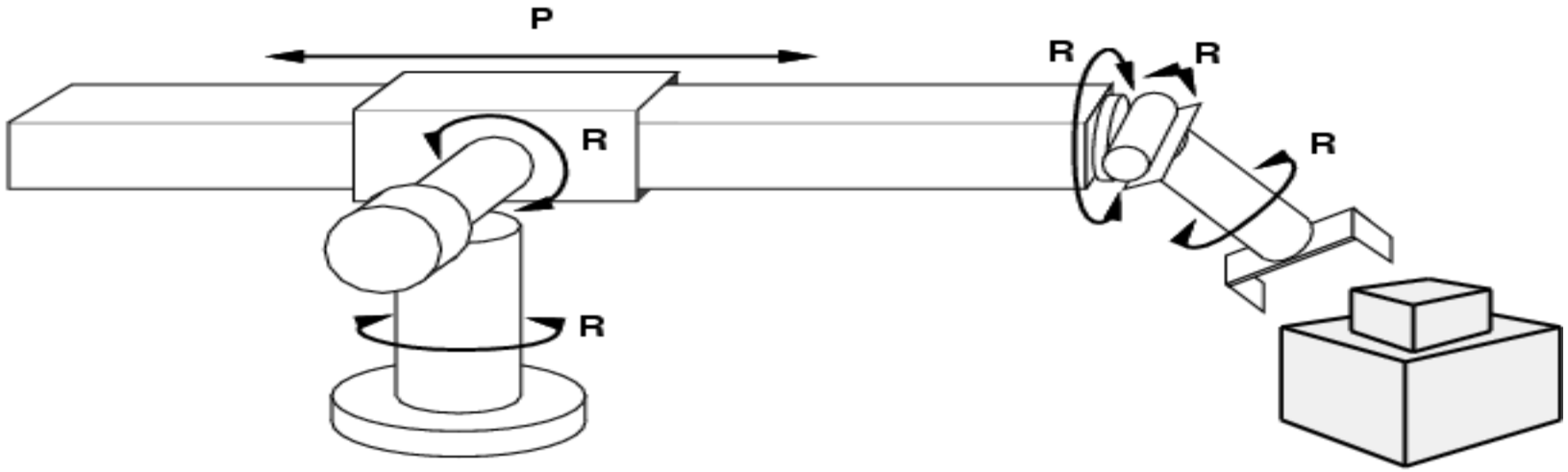
1	4	7
2	5	8
3	6	

Goal State

Example: The 8-puzzle Partial State Space



Example: Robot motion planning



- **States**
  - Real-valued joint parameters (angles, displacements)
- **Actions**
  - Continuous motions of robot joints
- **Goal state**
  - Configuration in which object is grasped
- **Path cost**
  - Time to execute, smoothness of path, etc.

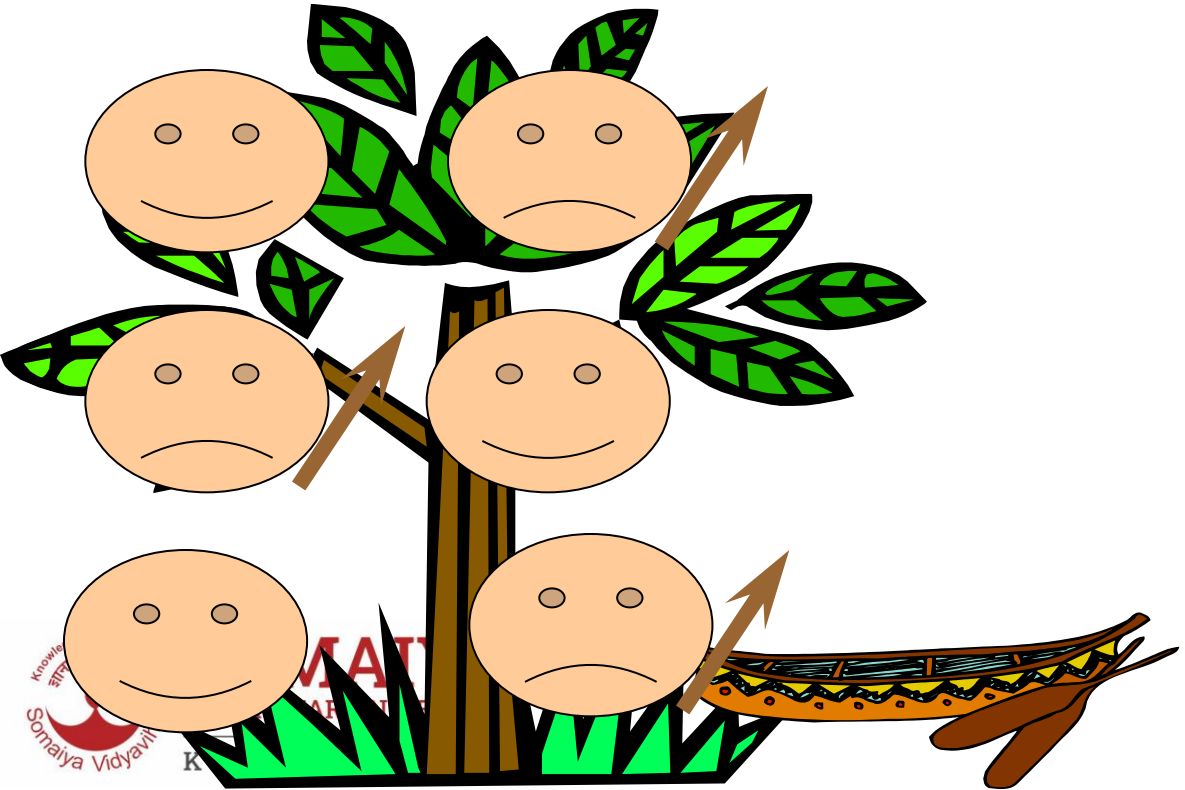
Example: Water Jug Problem

- You are given two jugs 4-gallon and 3-gallon. Neither has any measuring markers on it.
- You can fill the jugs up or empty them out from one to another or on ground.
- You need to measure out exactly two gallons of water into 4-gallon jug.

Give a complete problem formulation

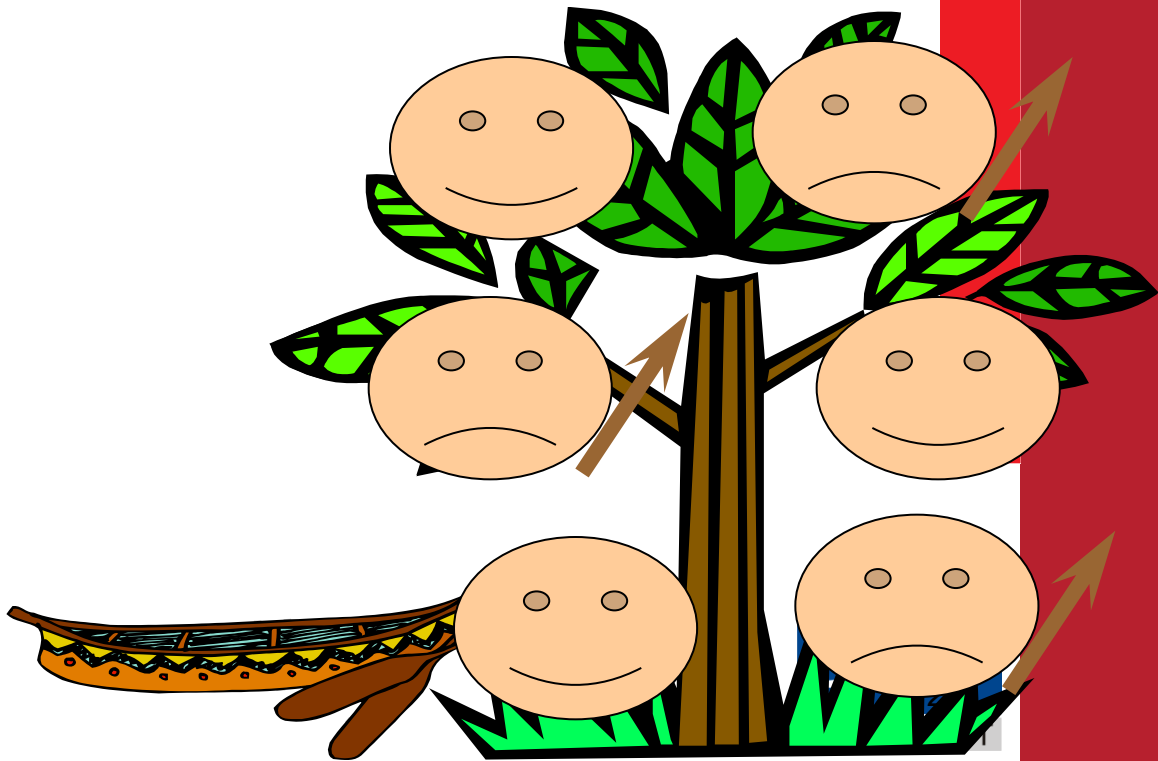
Missionaries and Cannibals

Initial State



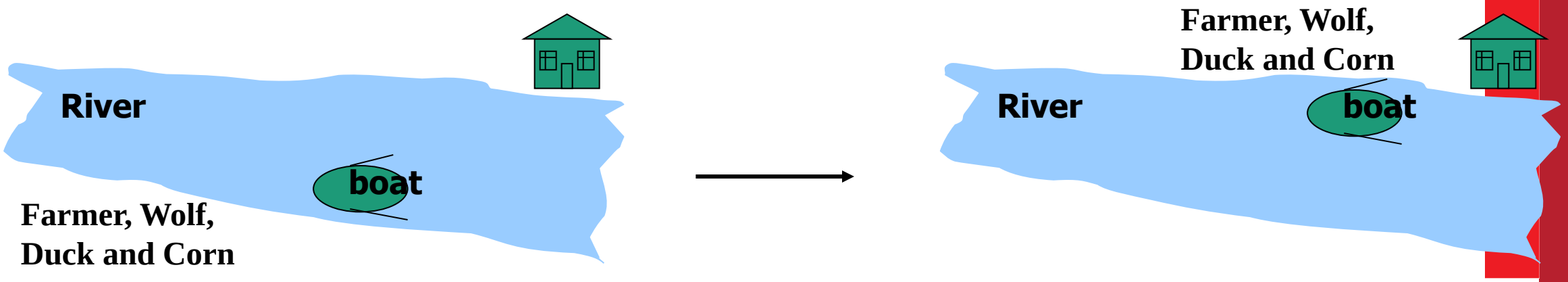
Missionaries and Cannibals

Goal State



The River Problem

A farmer wishes to carry a wolf, a duck and corn across a river, from the south to the north shore. The farmer is the proud owner of a small rowing boat called Bounty which he feels is easily up to the job. Unfortunately the boat is only large enough to carry at most the farmer and one other item. Worse again, if left unattended the wolf will eat the duck and the duck will eat the corn.



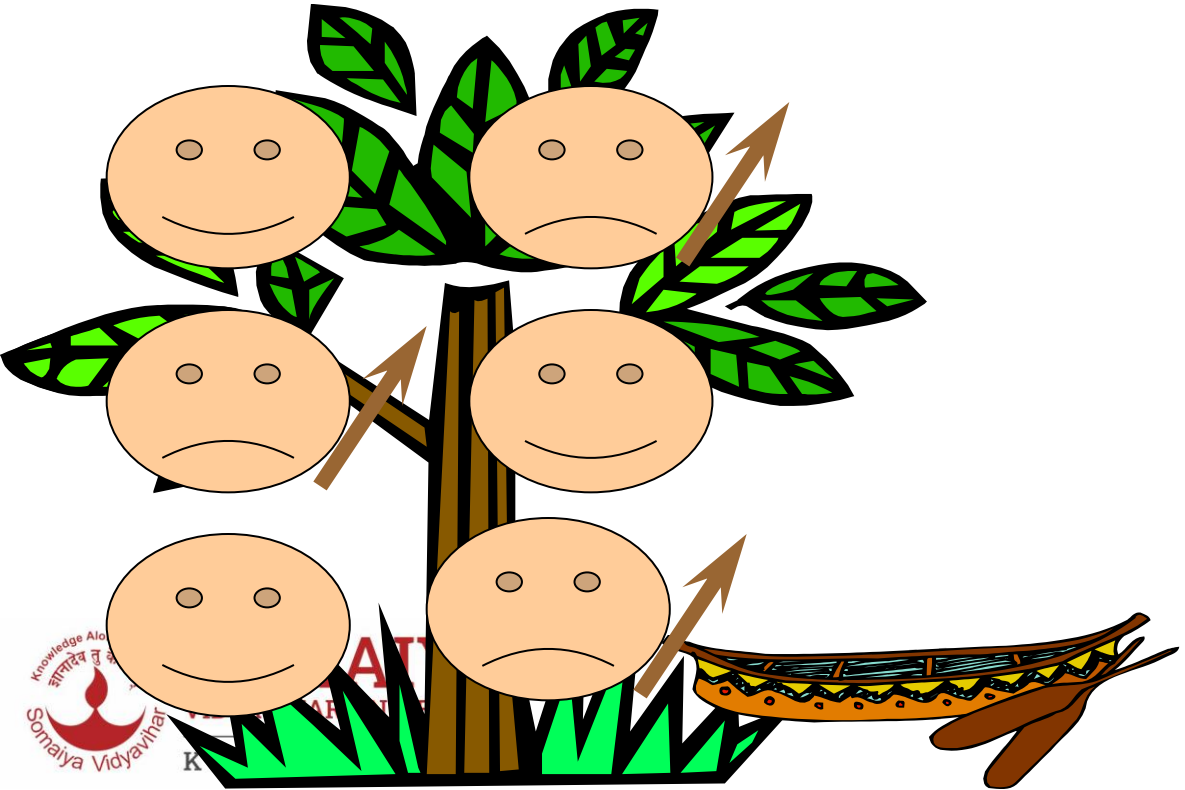
How can the farmer safely transport the wolf, the duck and the corn to the opposite shore?

Missionaries and cannibals

- States: three numbers (i,j,k) representing the number of missionaries, cannibals, and canoes on the left bank of the river.
- Initial state: (3, 3, 1)
- Operators: take one missionary, one cannibal, two missionaries, two cannibals, one missionary and one cannibal across the river in a given direction (I.e. ten operators).
- Goal Test: reached state (0, 0, 0)
- Path Cost: Number of crossings.

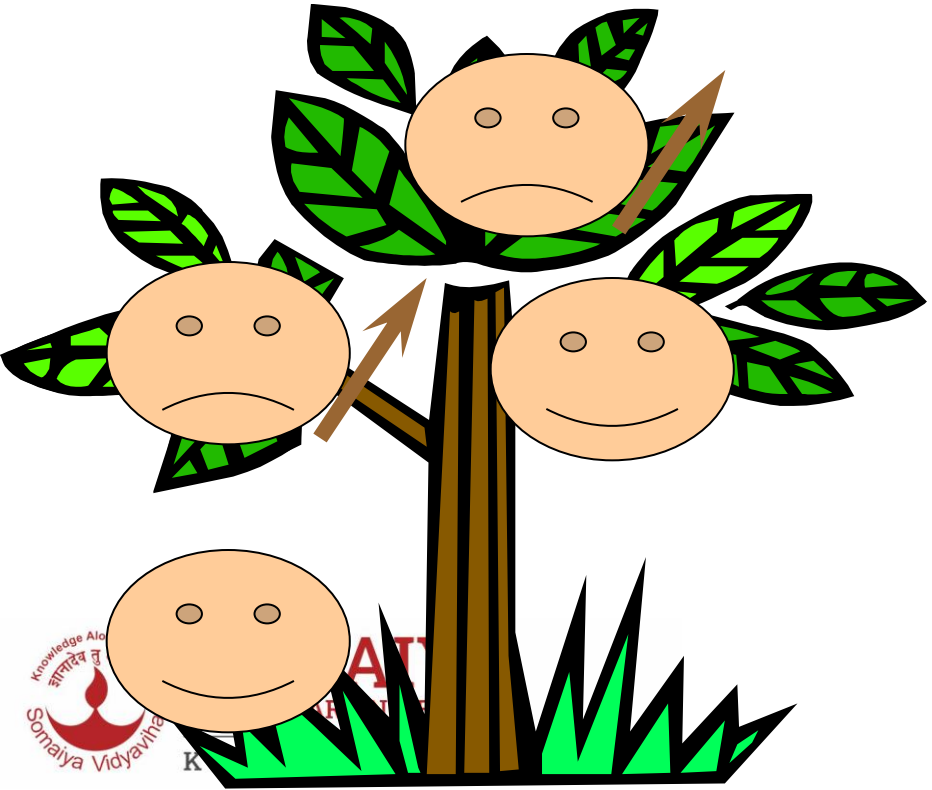
Missionaries and Cannibals

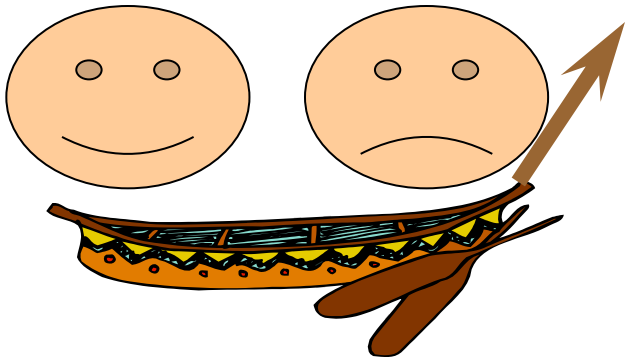
(3,3,1): Initial State



Missionaries and Cannibals

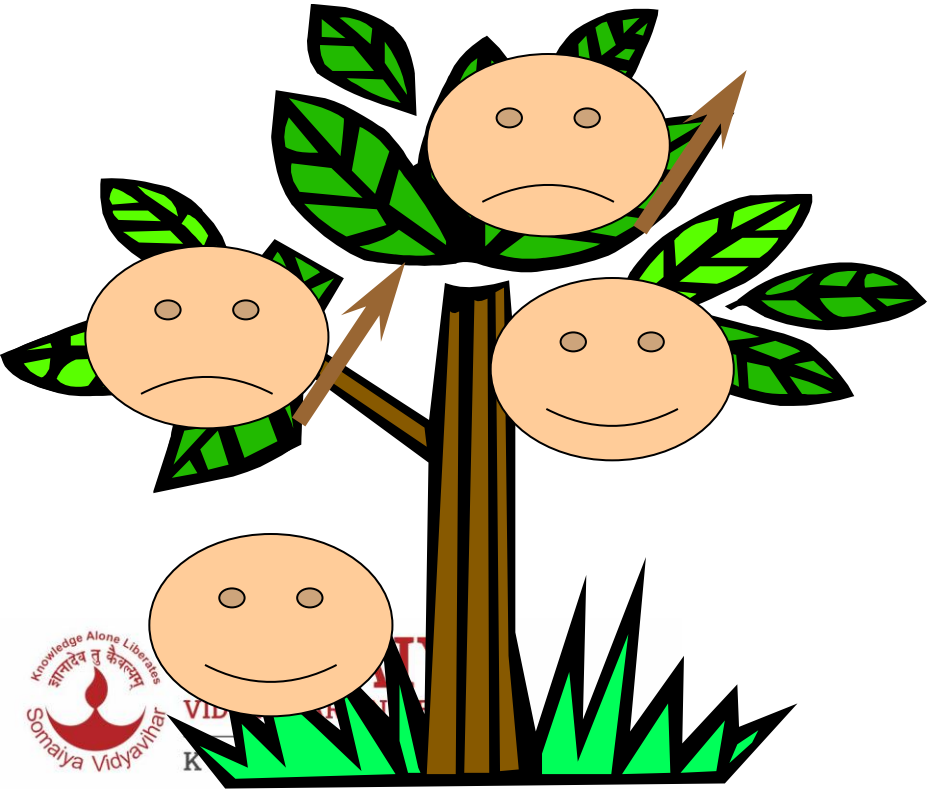
A missionary and cannibal cross

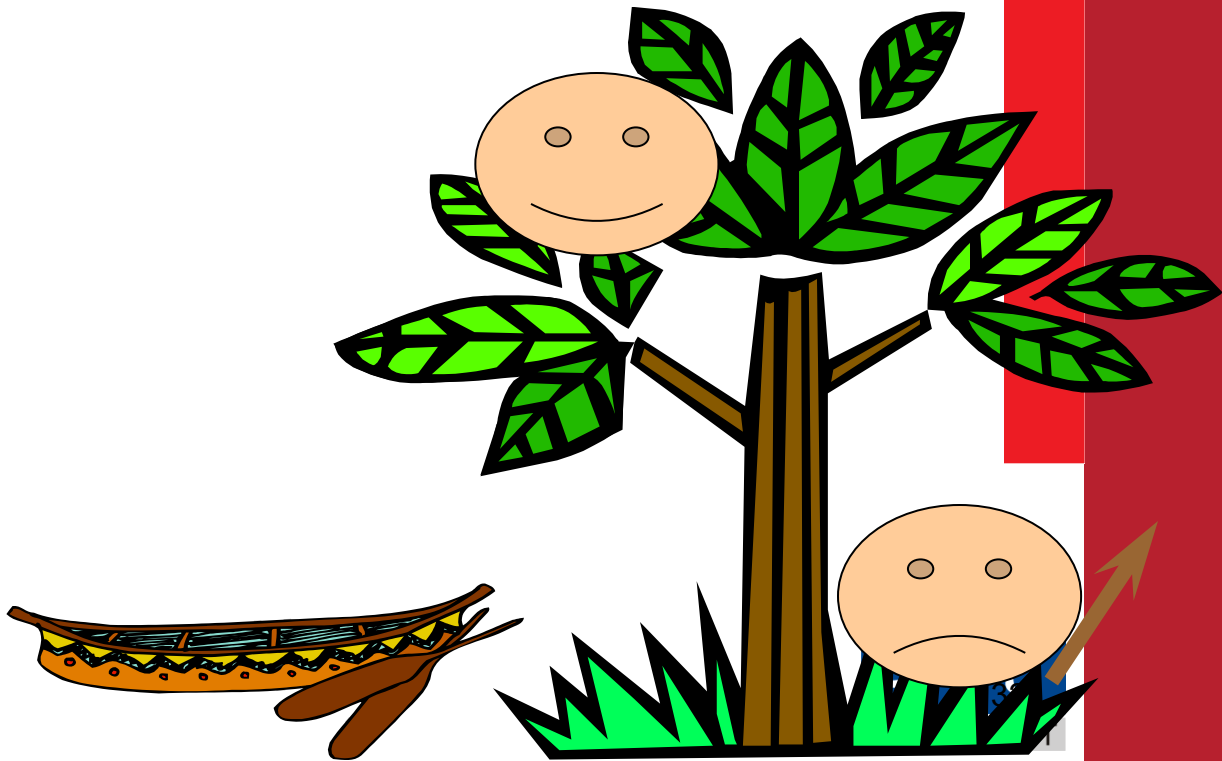




Missionaries and Cannibals

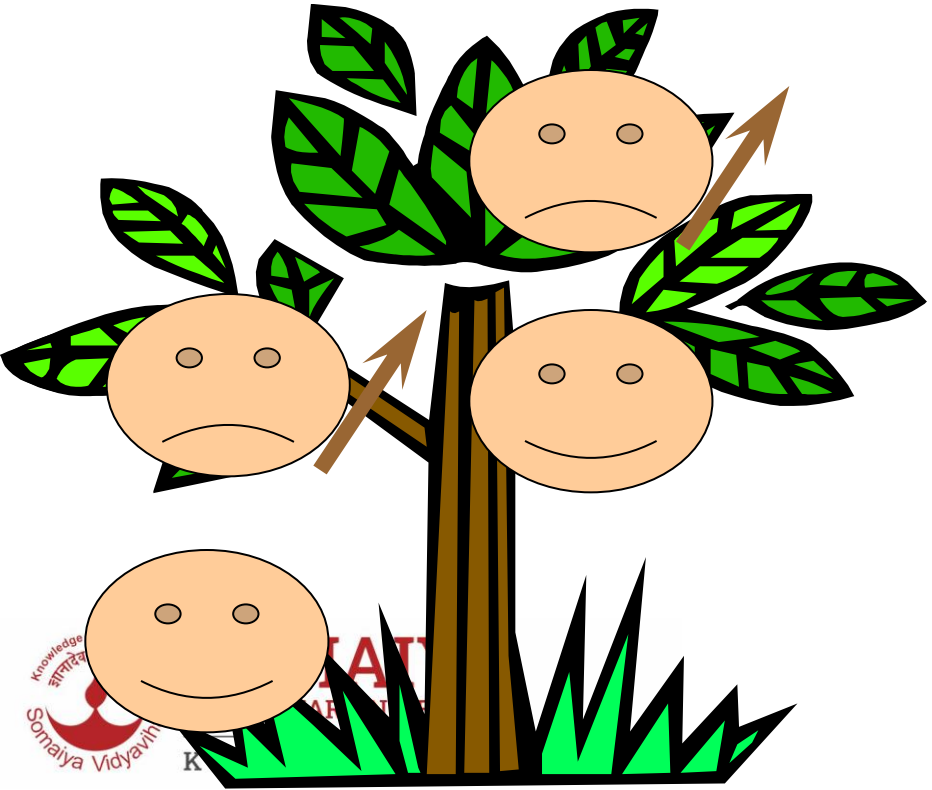
(2,2,0)

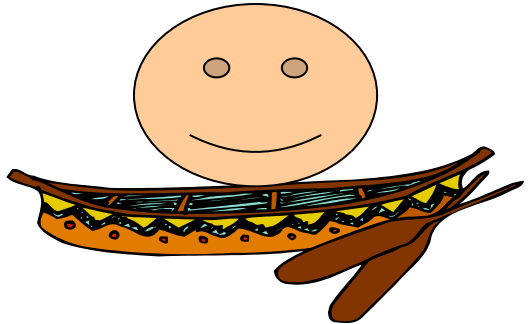


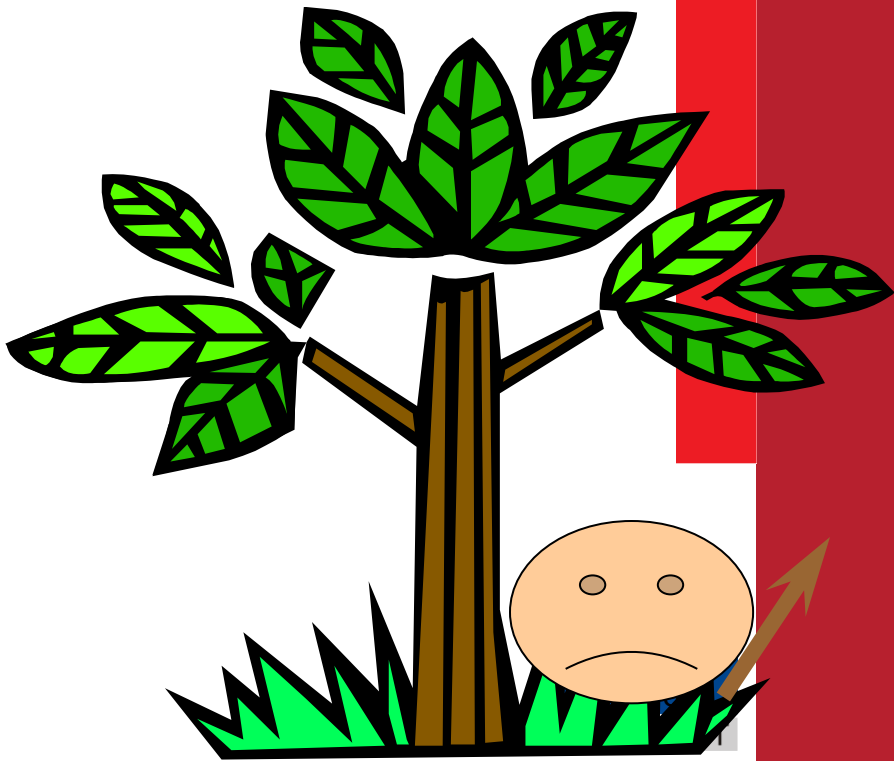


Missionaries and Cannibals

One missionary returns

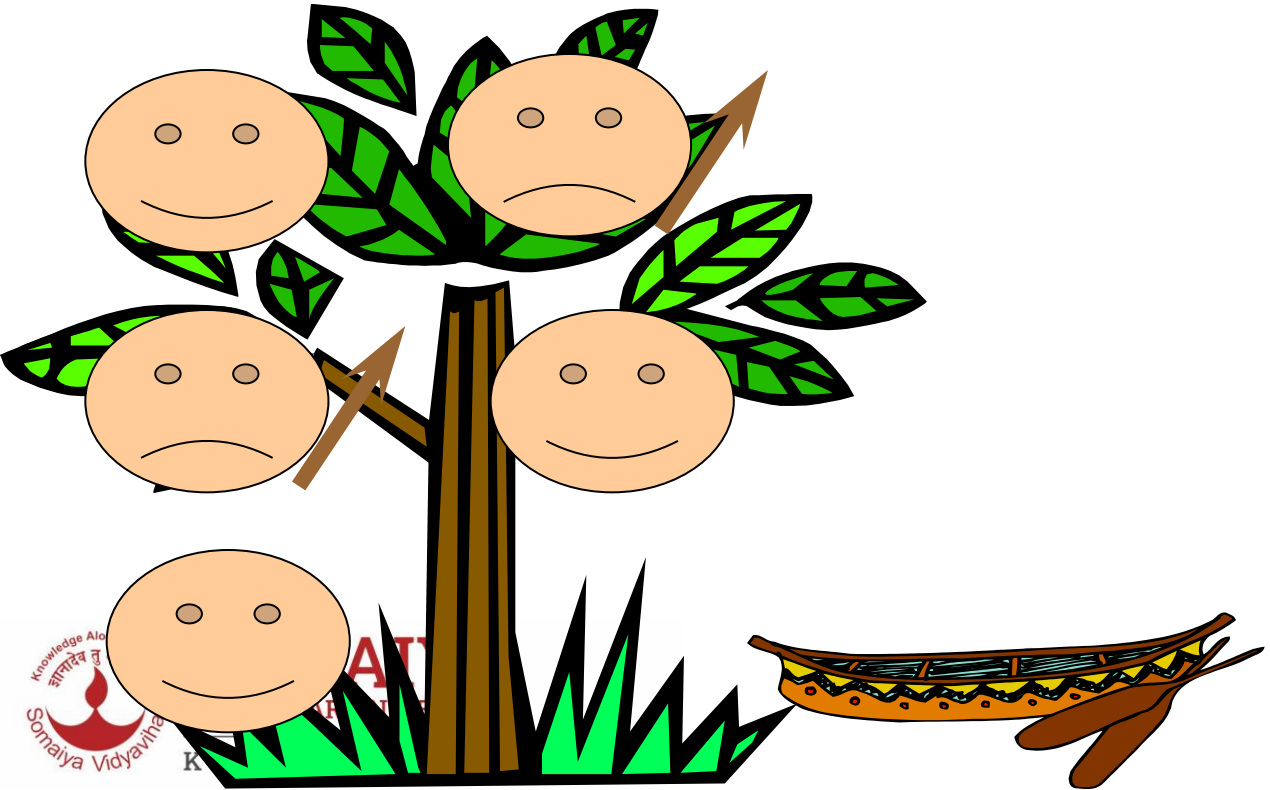


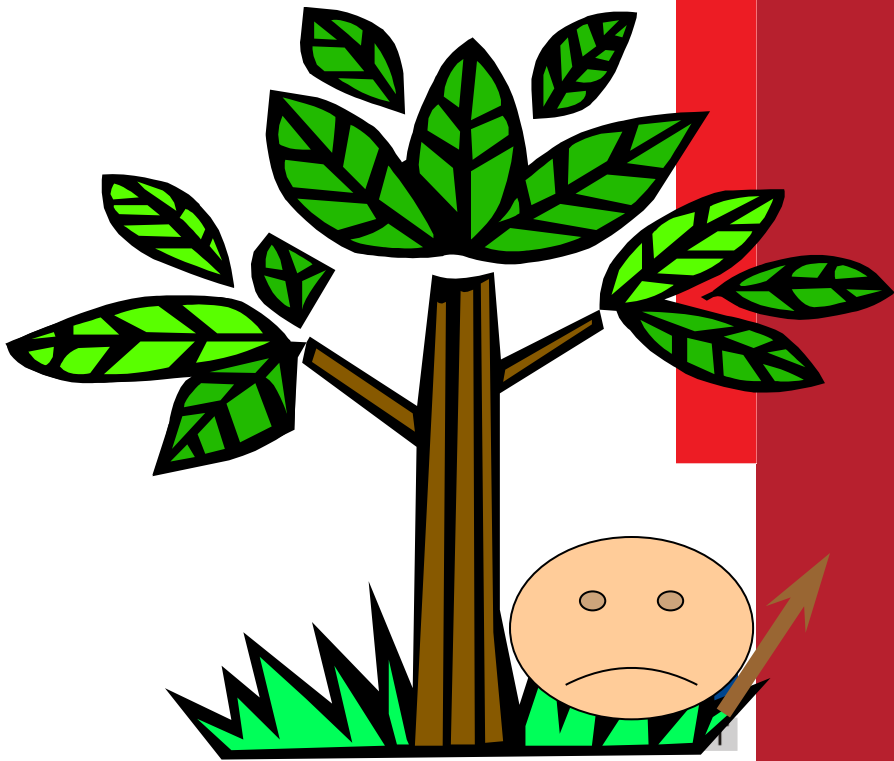




Missionaries and Cannibals

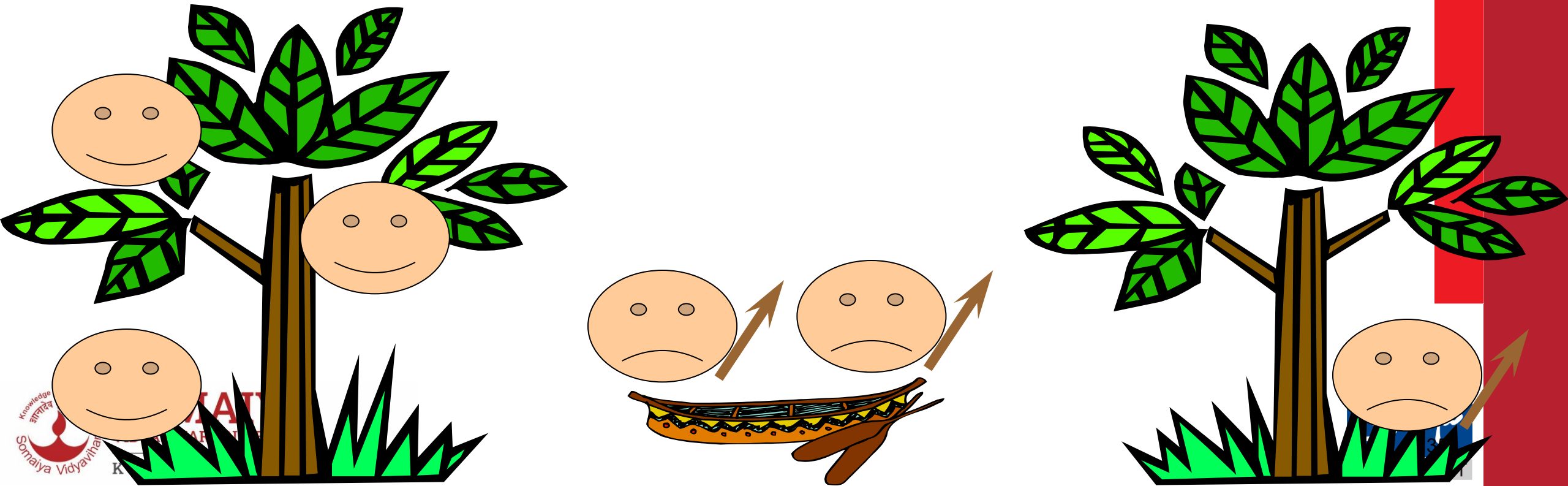
(3,2,1)





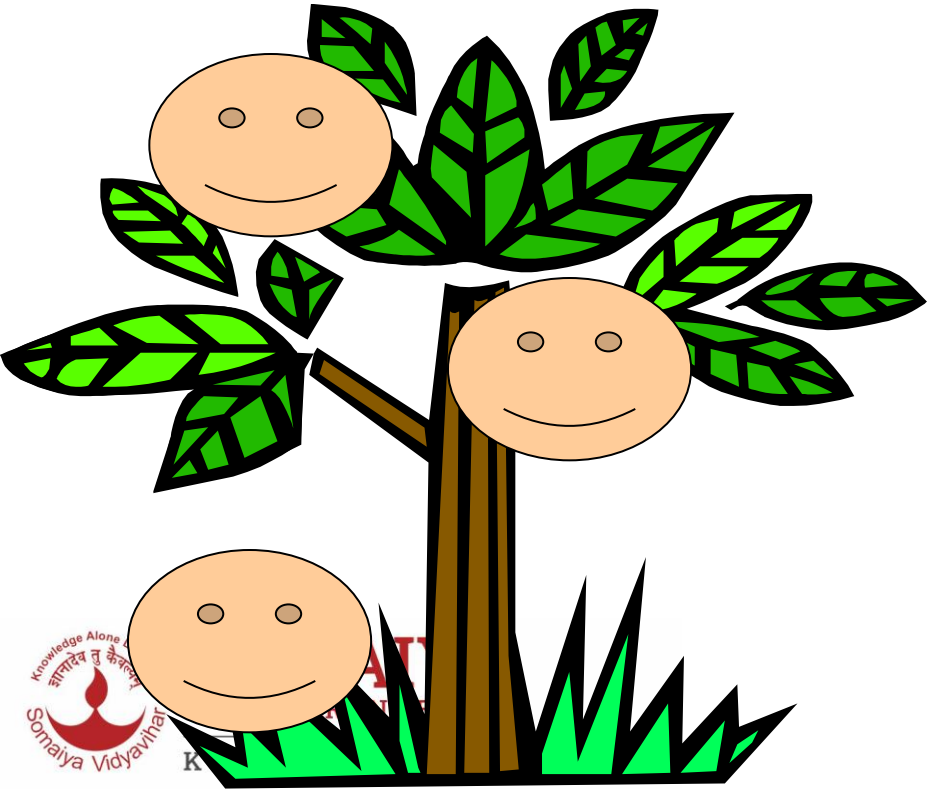
Missionaries and Cannibals

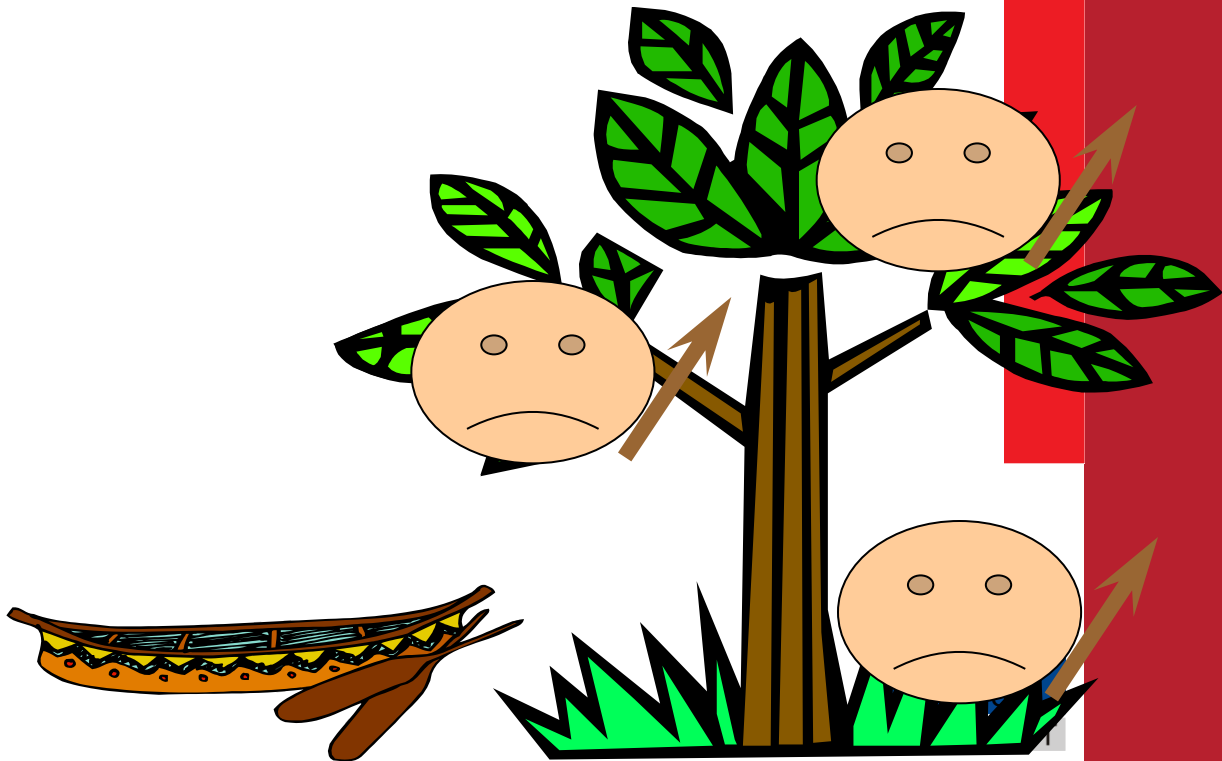
Two cannibals cross



Missionaries and Cannibals

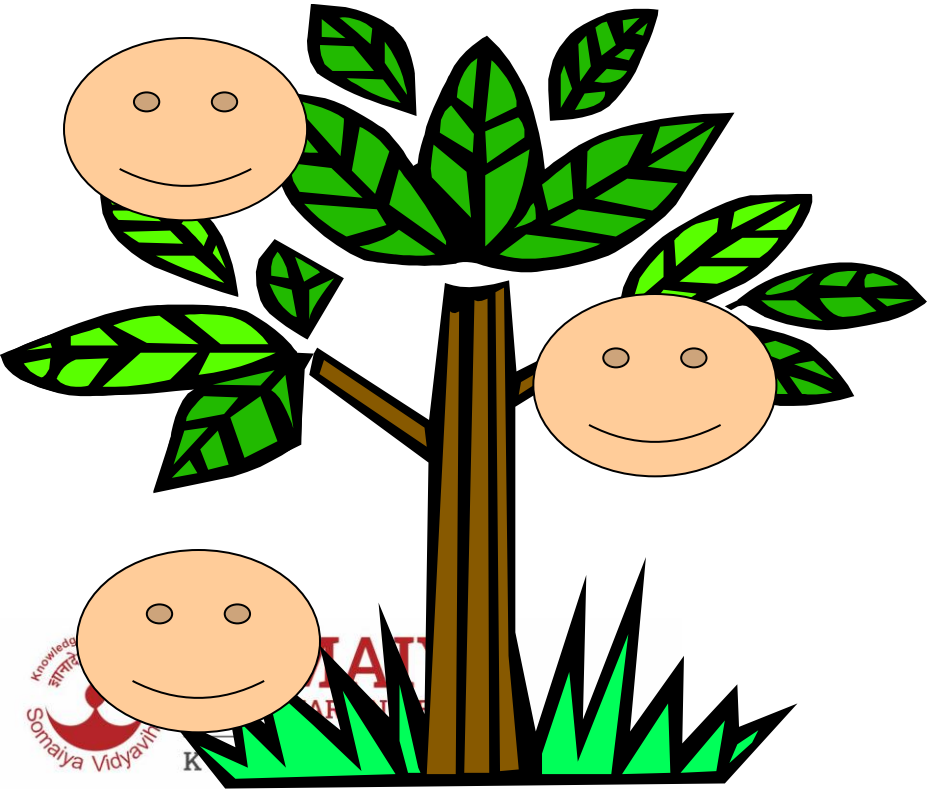
(3,0,0)

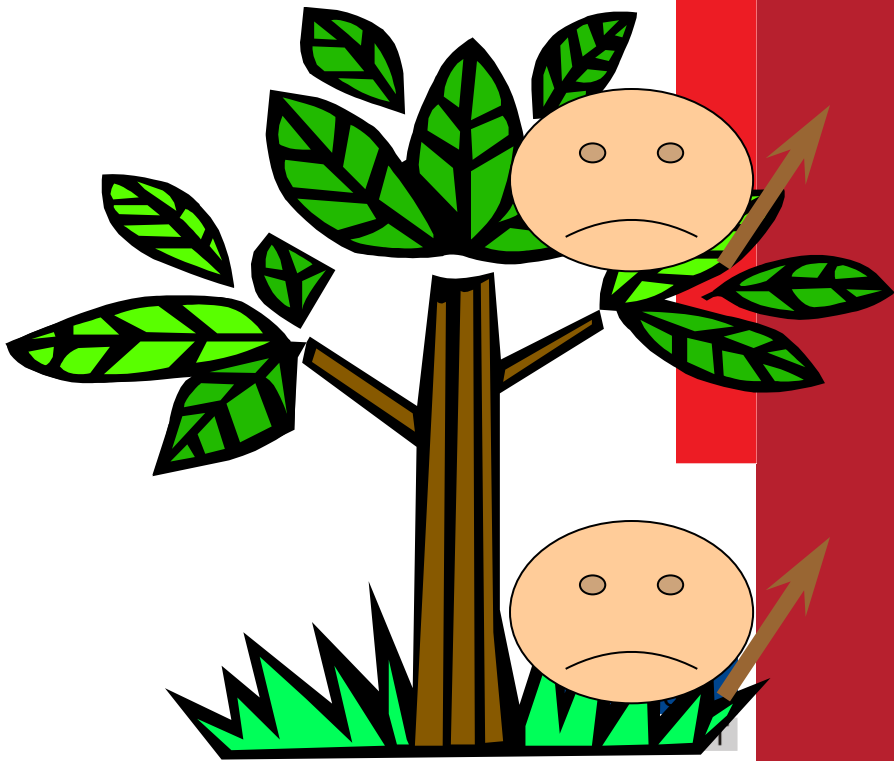




Missionaries and Cannibals

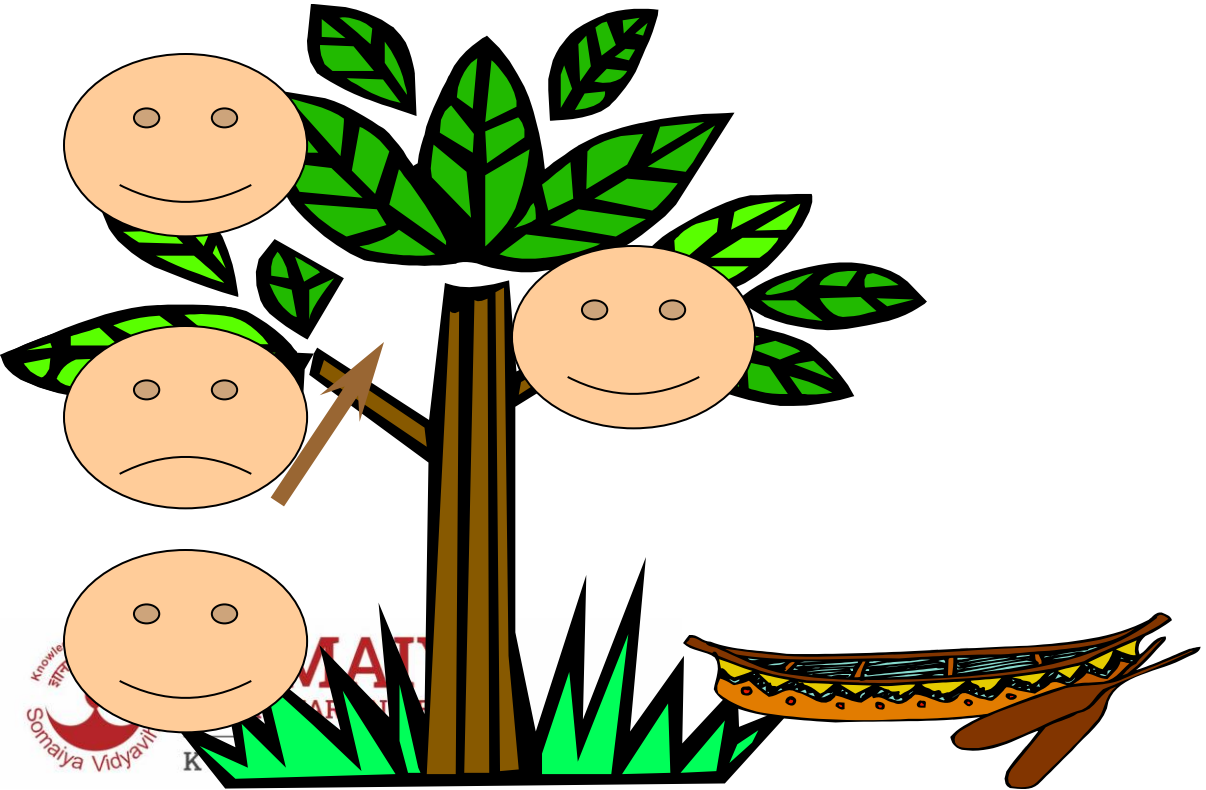
A cannibal returns

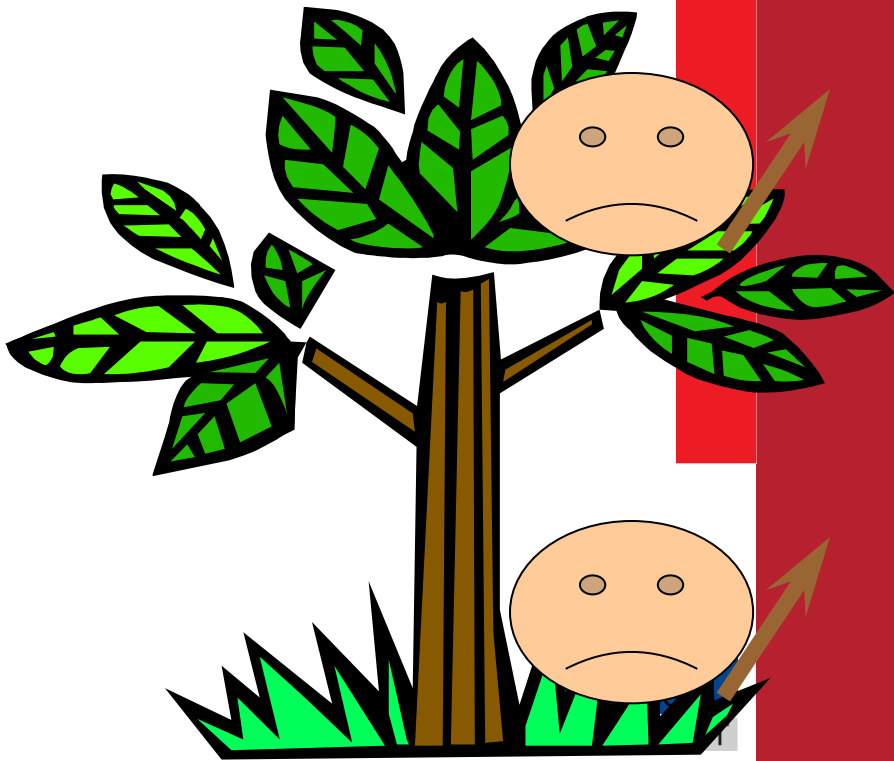




Missionaries and Cannibals

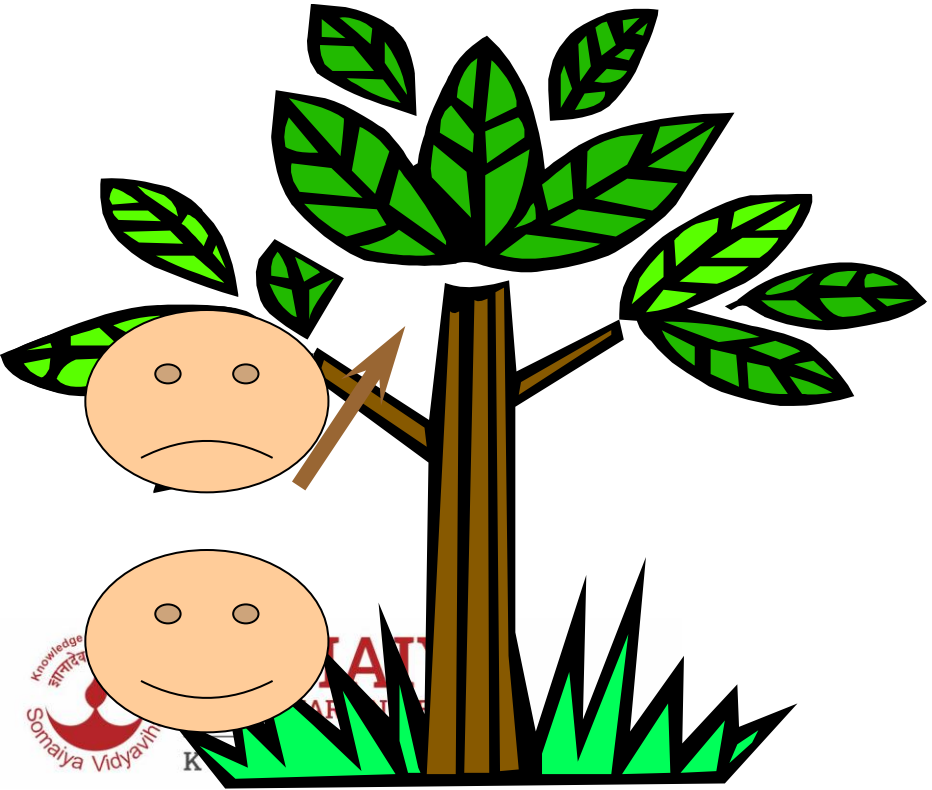
(3,1,1)

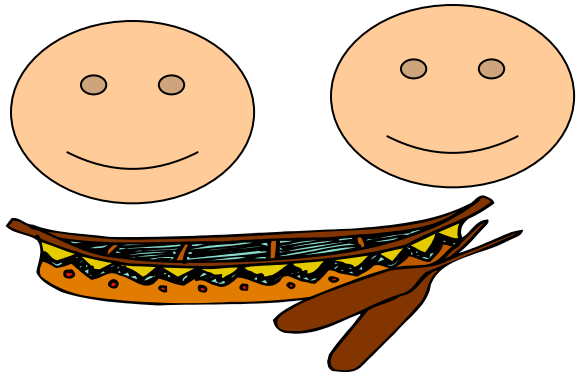


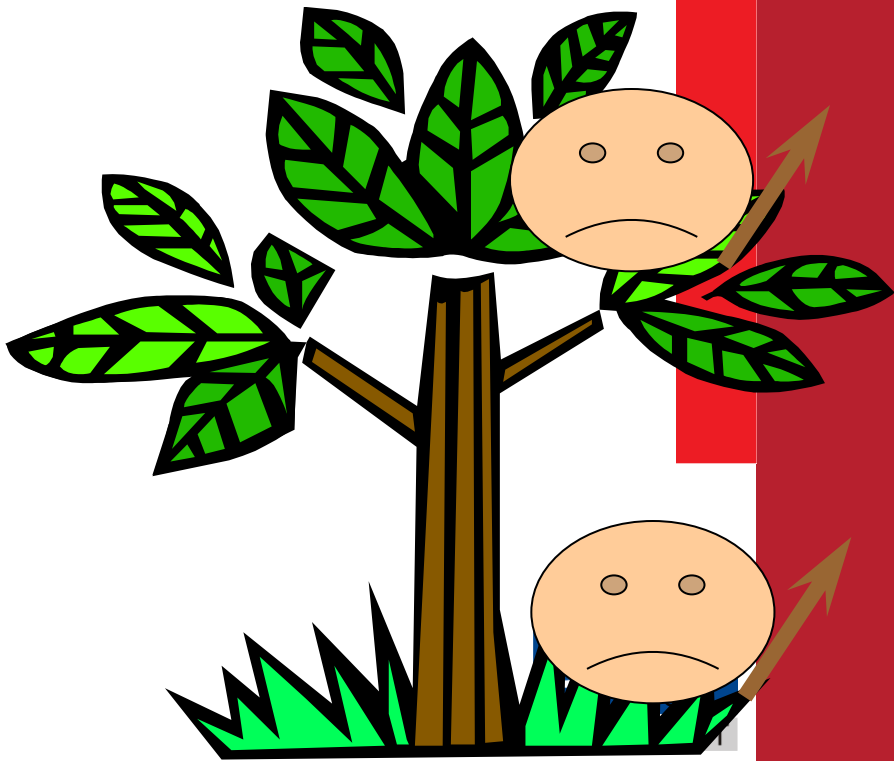


Missionaries and Cannibals

Two missionaries cross

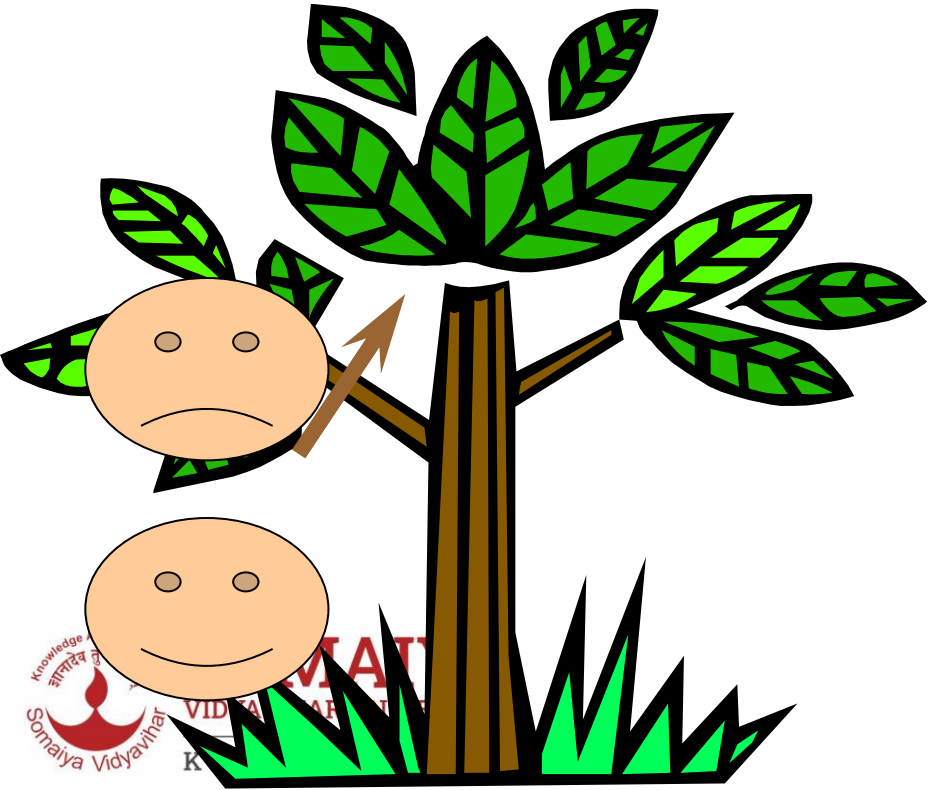


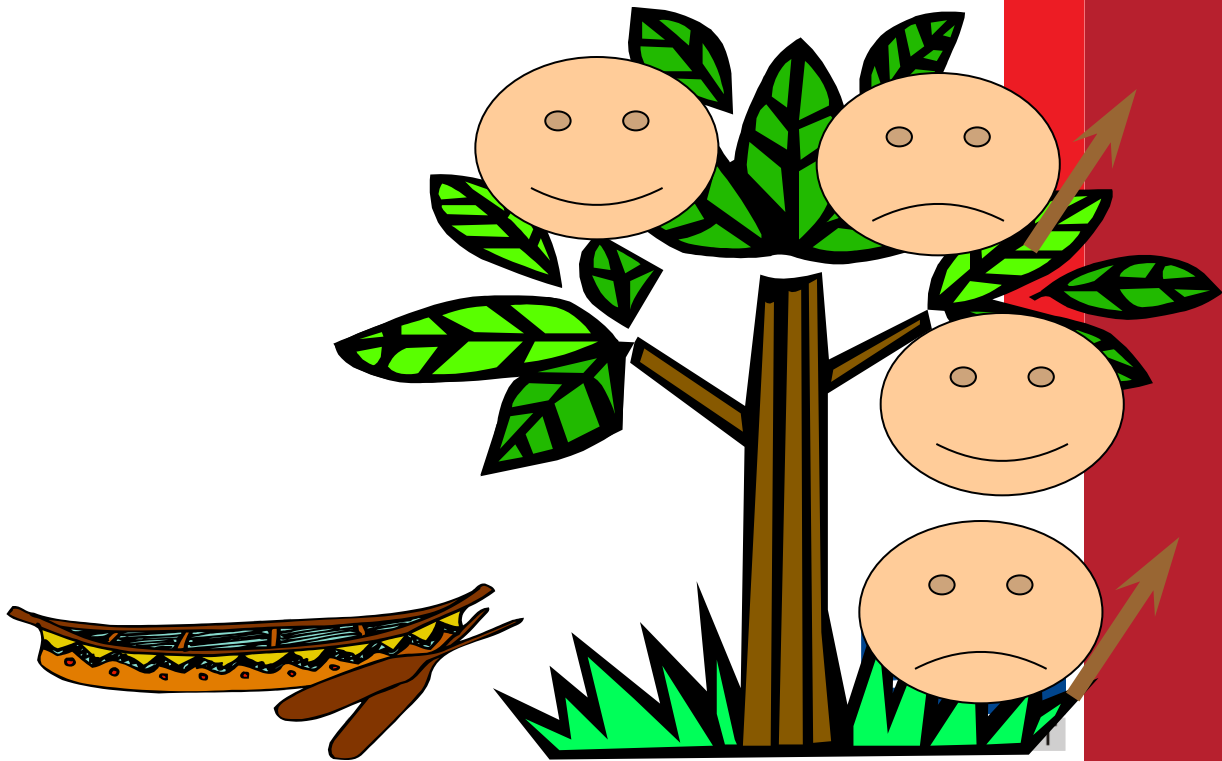




Missionaries and Cannibals

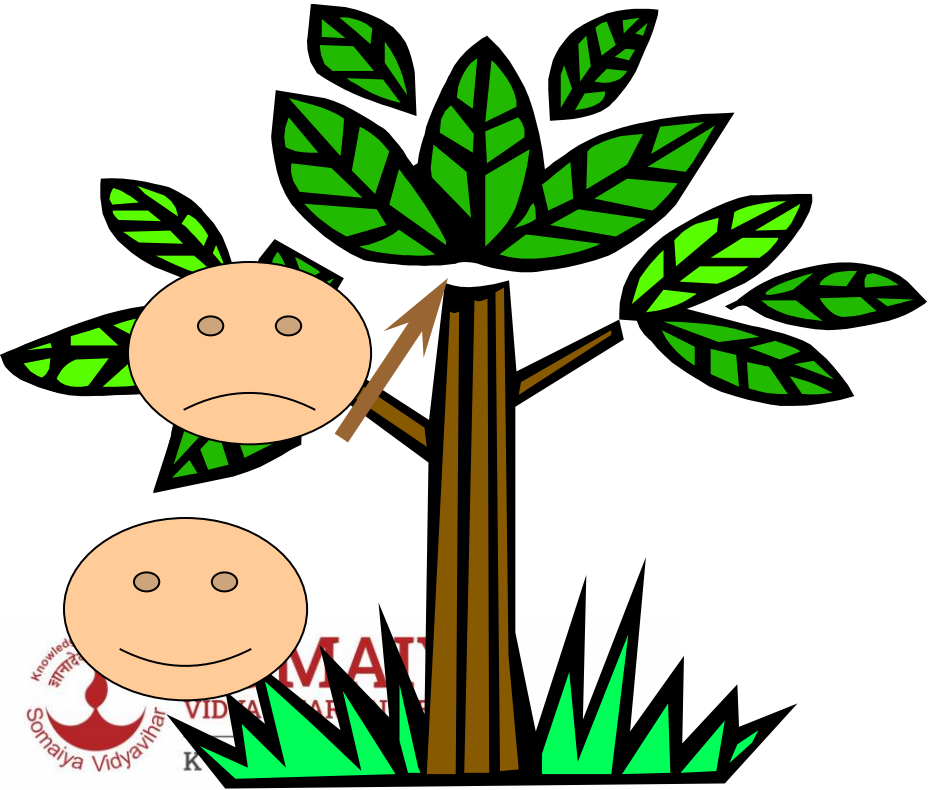
(1,1,0)

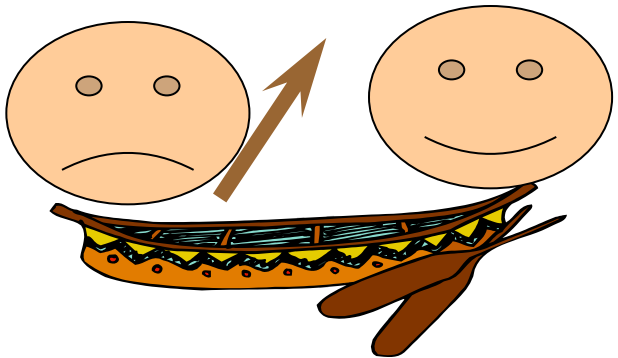


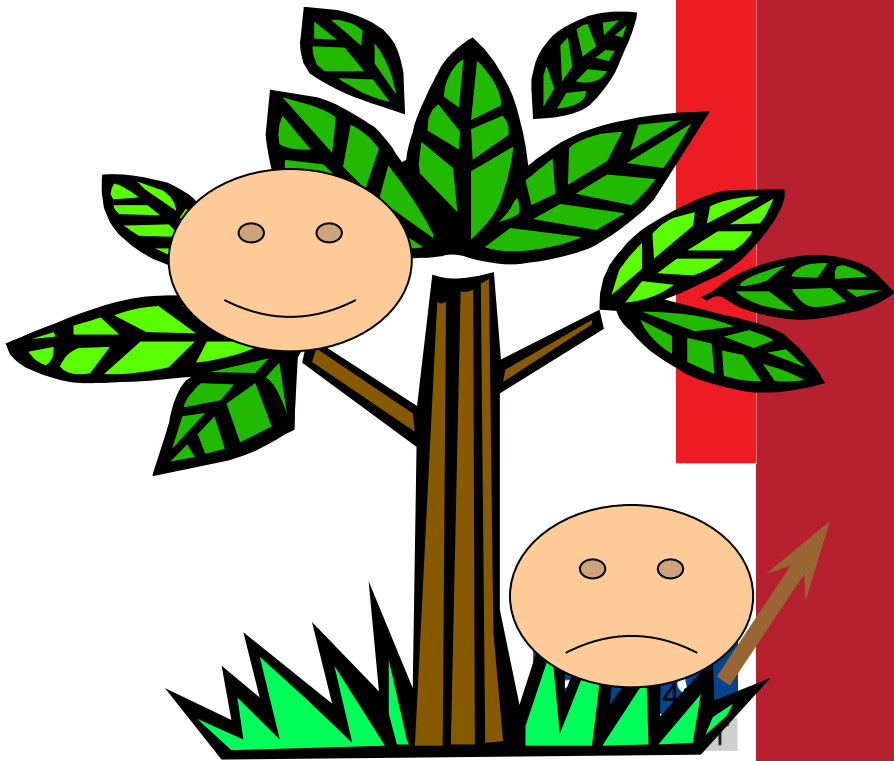


Missionaries and Cannibals

A missionary and cannibal return

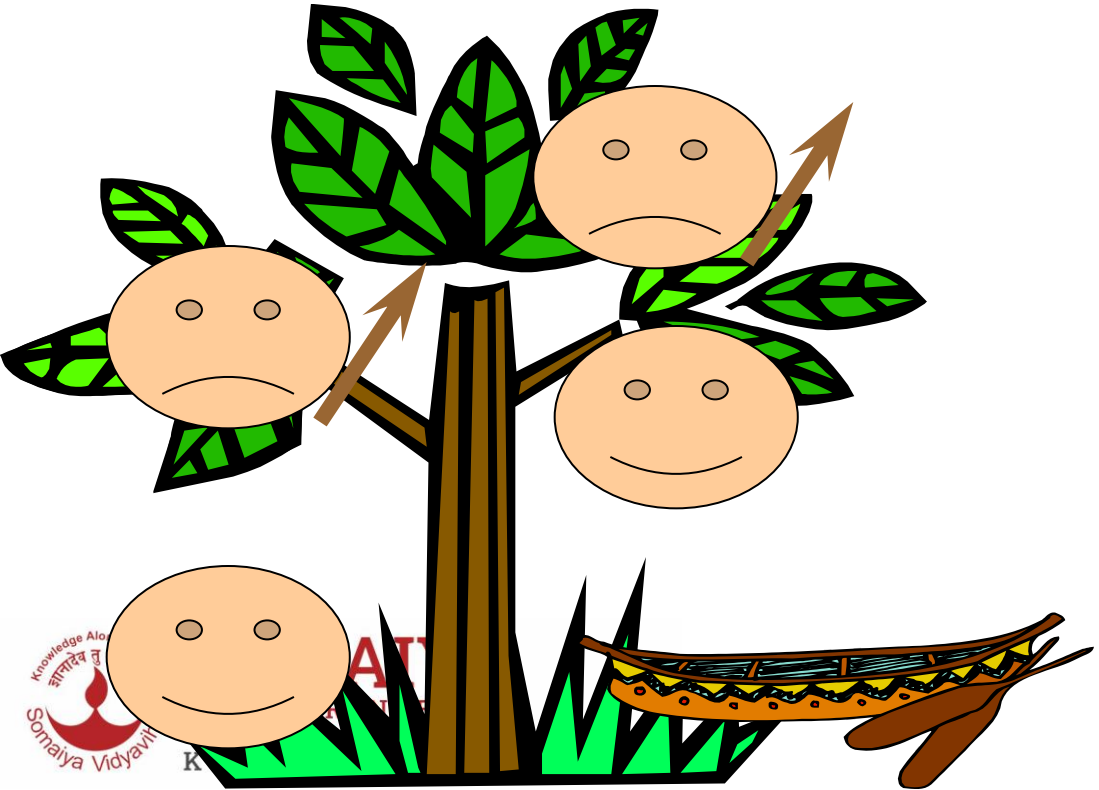


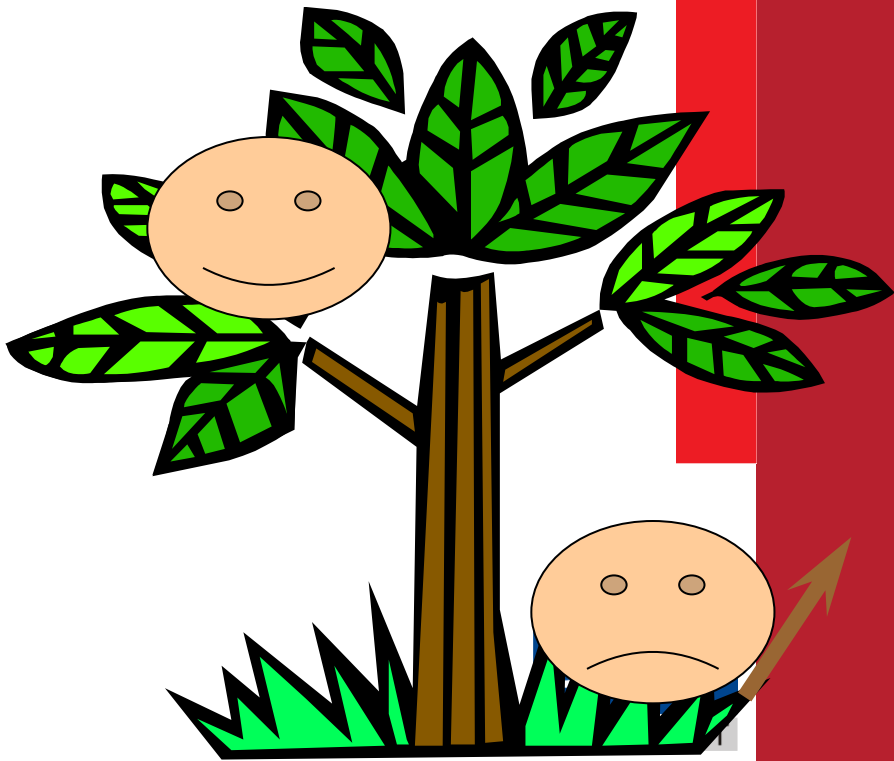




Missionaries and Cannibals

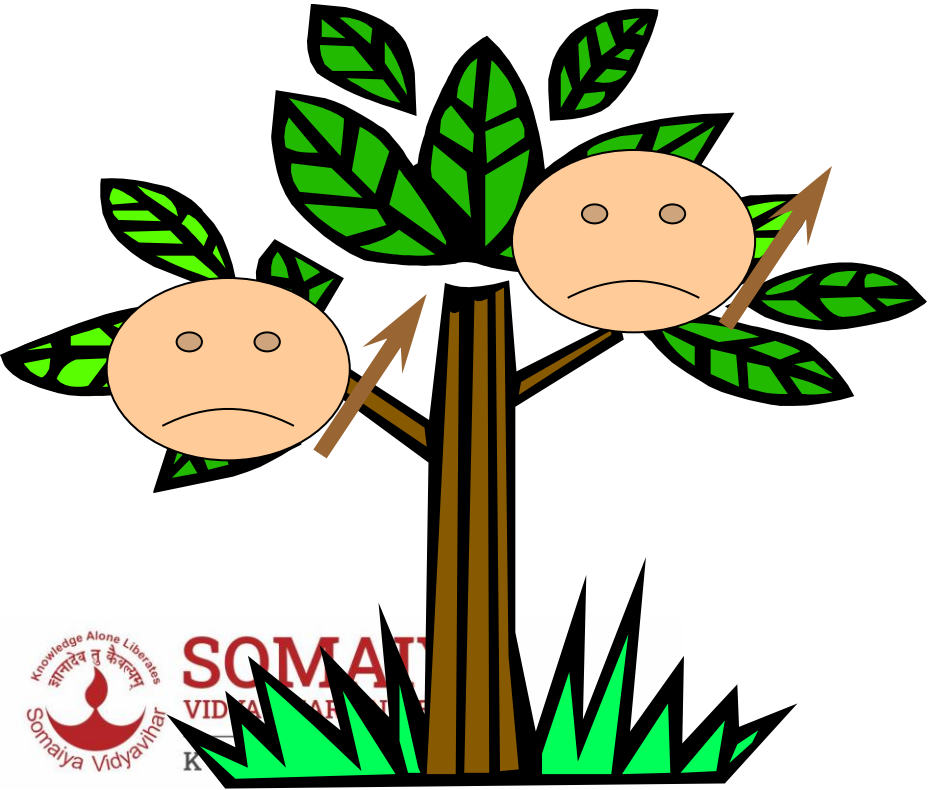
(2,2,1)

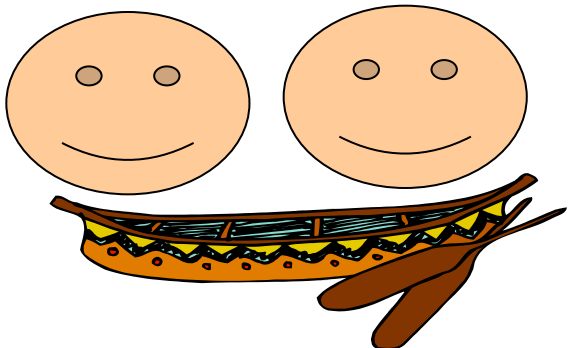


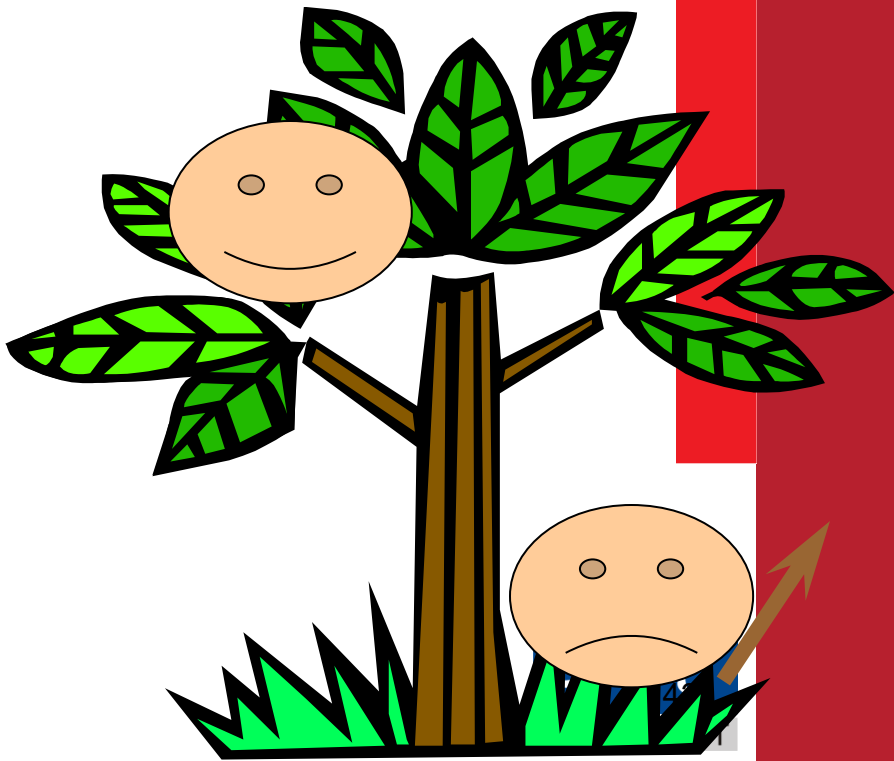


Missionaries and Cannibals

Two Missionaries cross

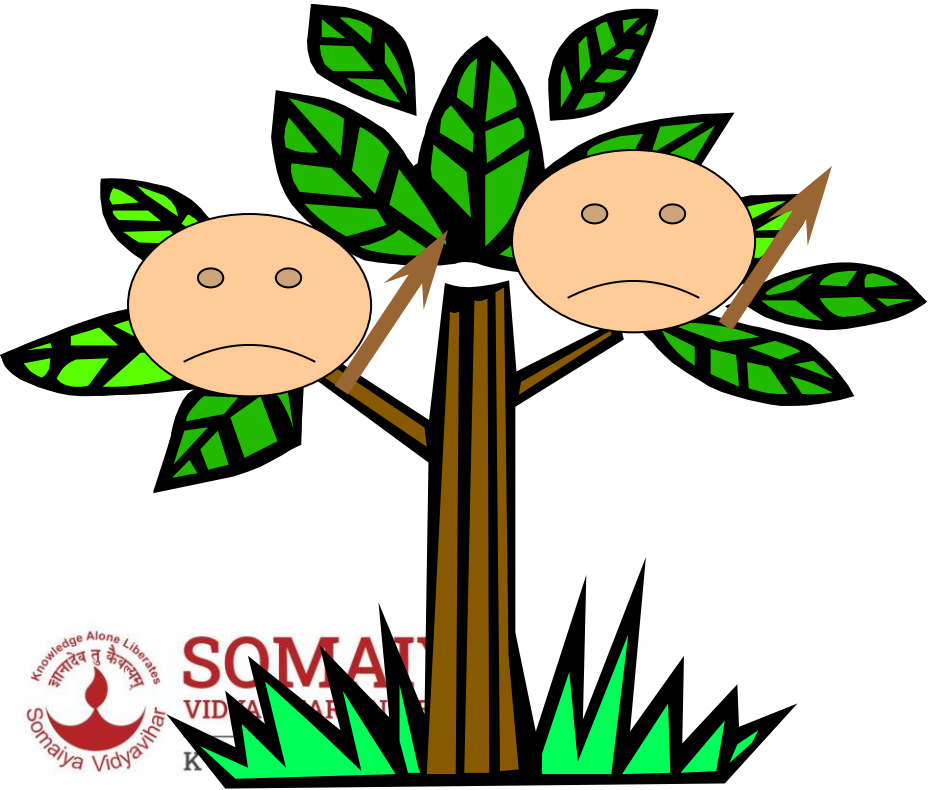


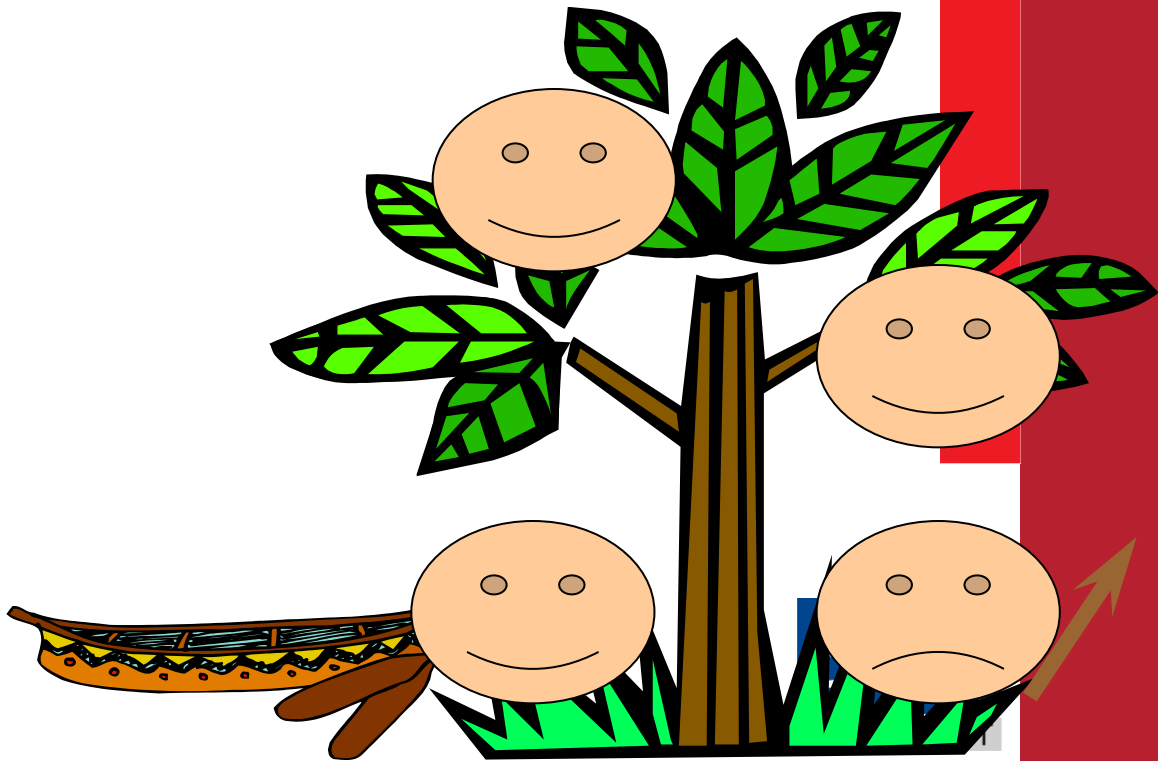




Missionaries and Cannibals

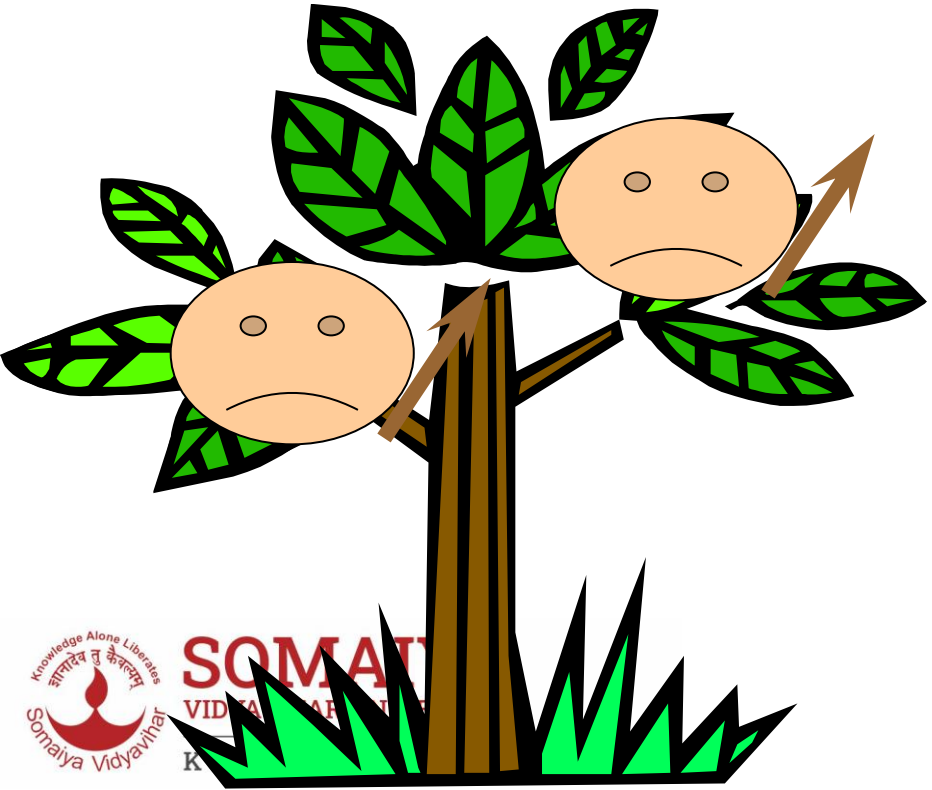
(0,2,0)

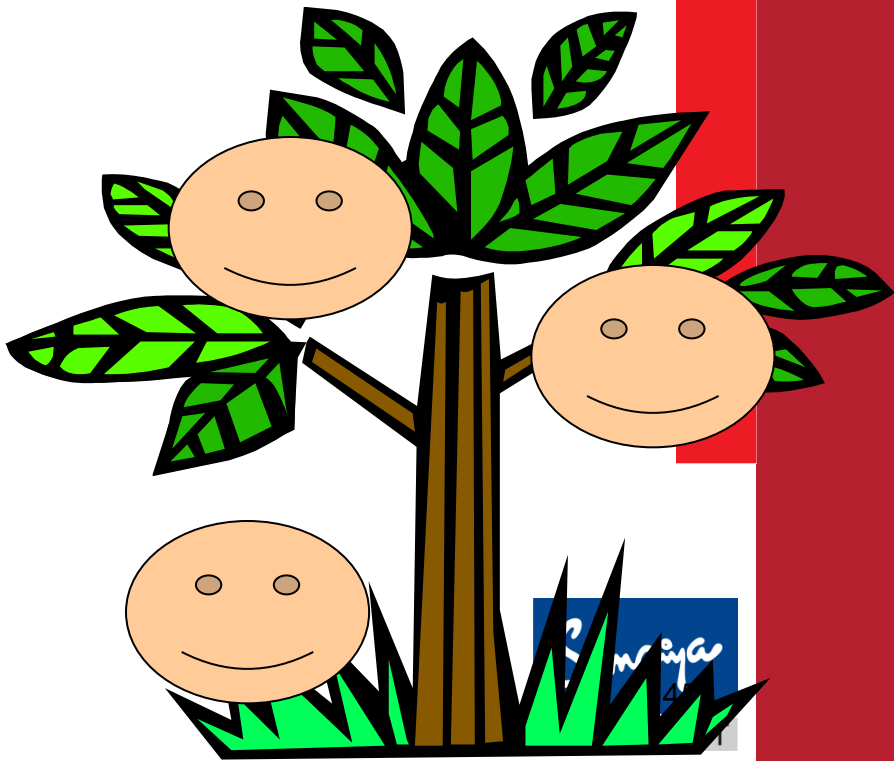




Missionaries and Cannibals

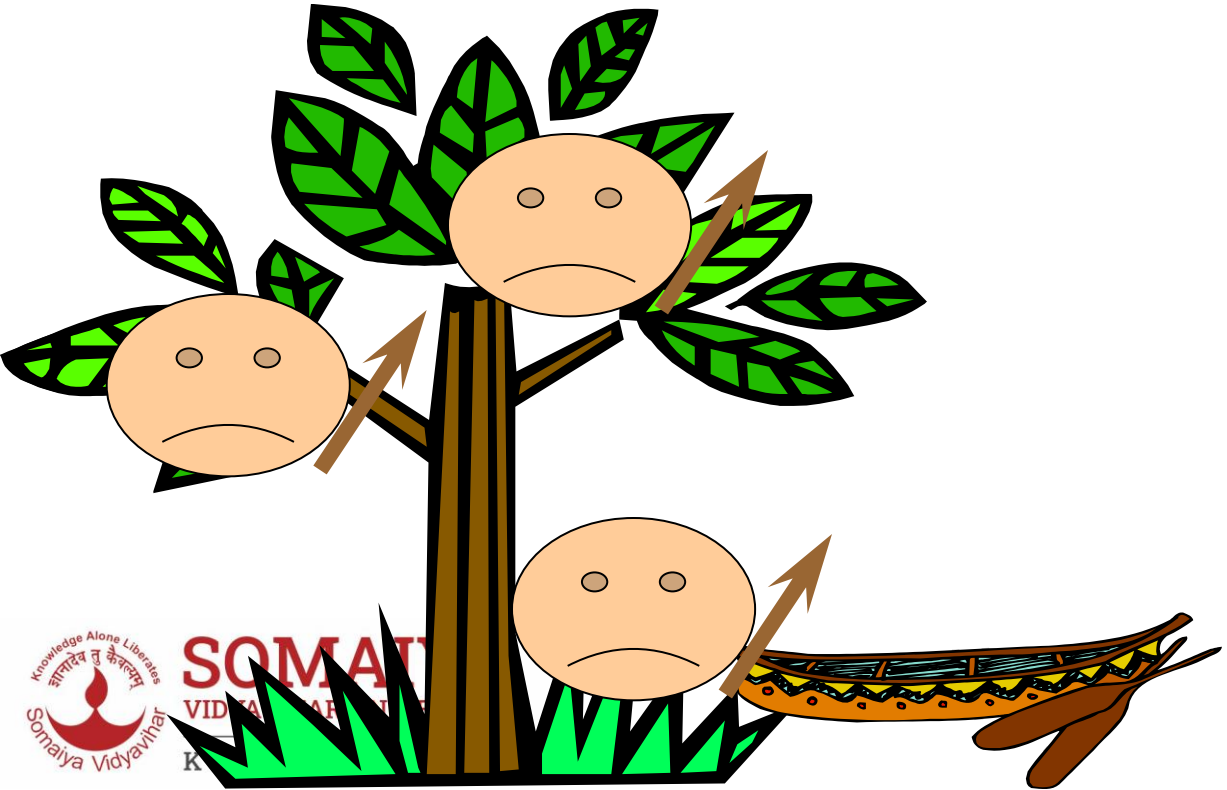
A cannibal returns

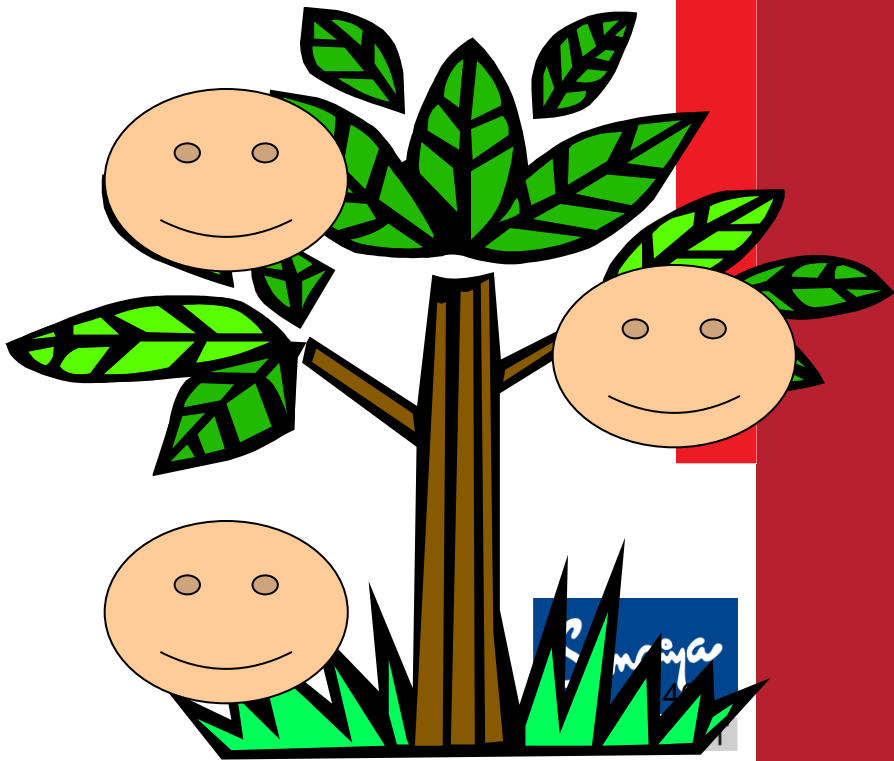




Missionaries and Cannibals

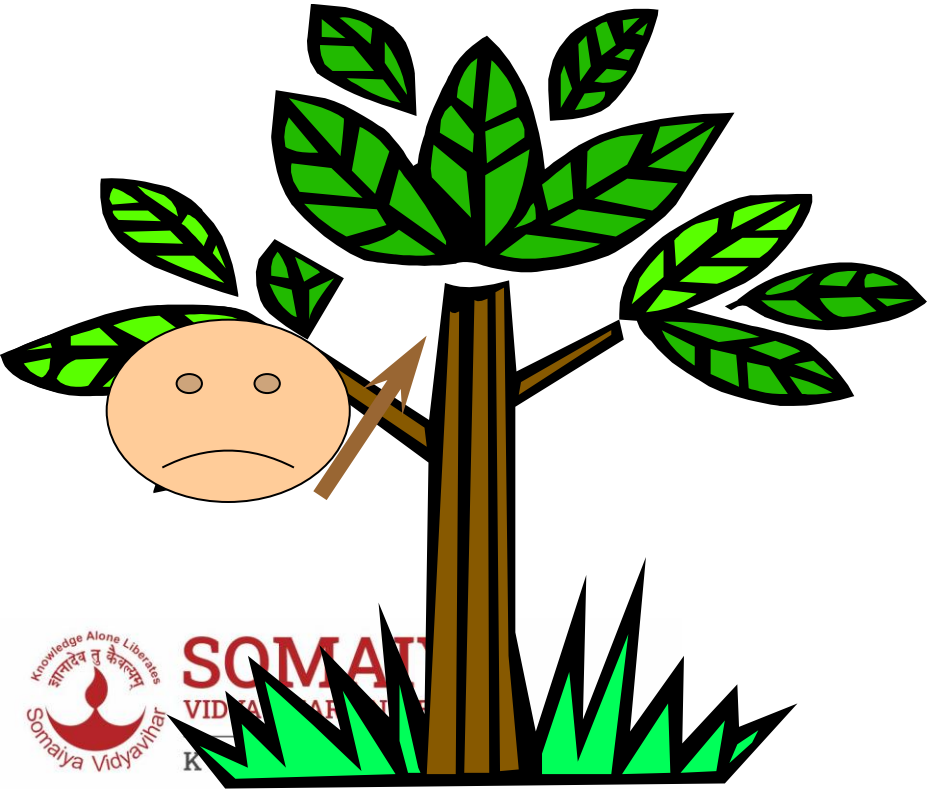
(0,3,1)

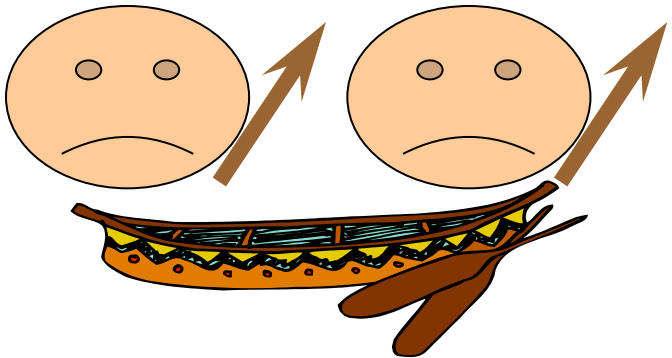


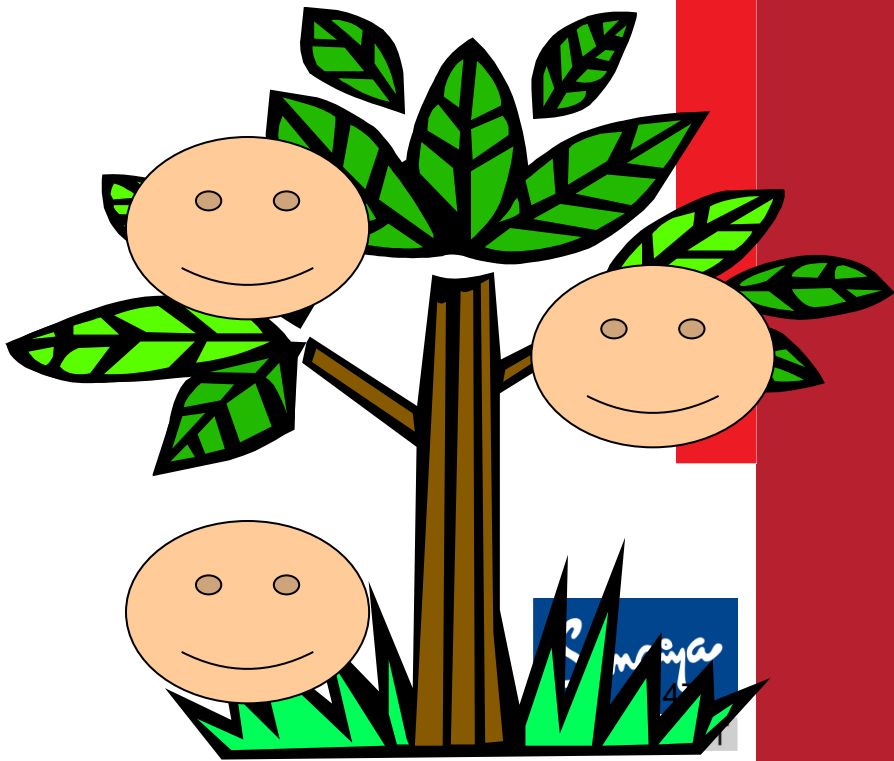


Missionaries and Cannibals

Two cannibals cross

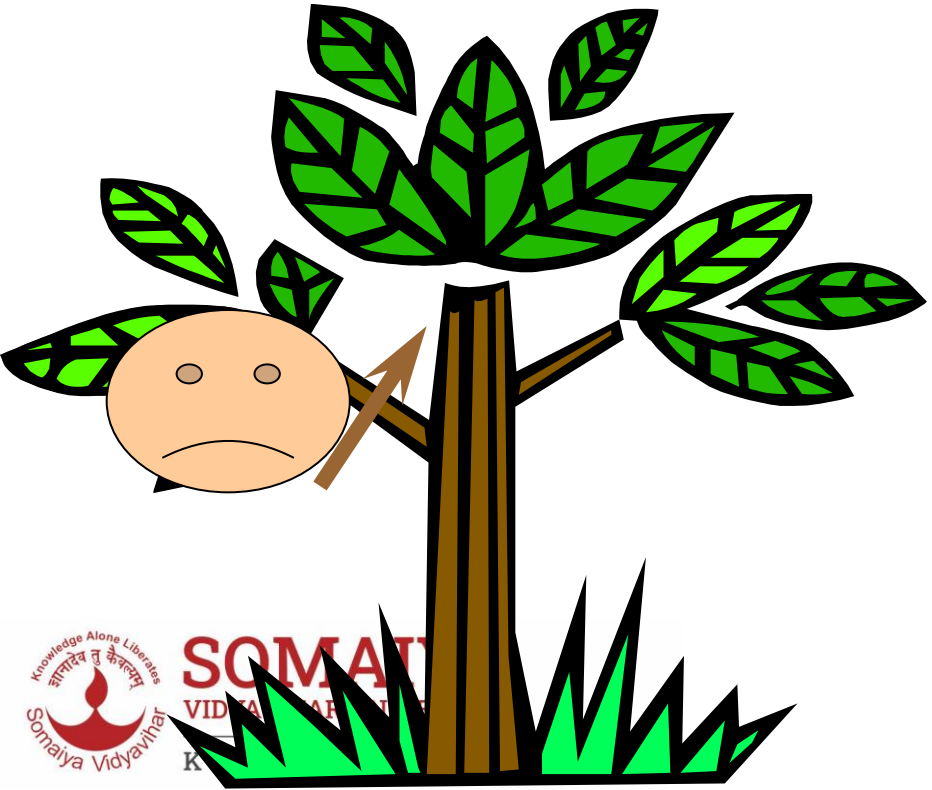


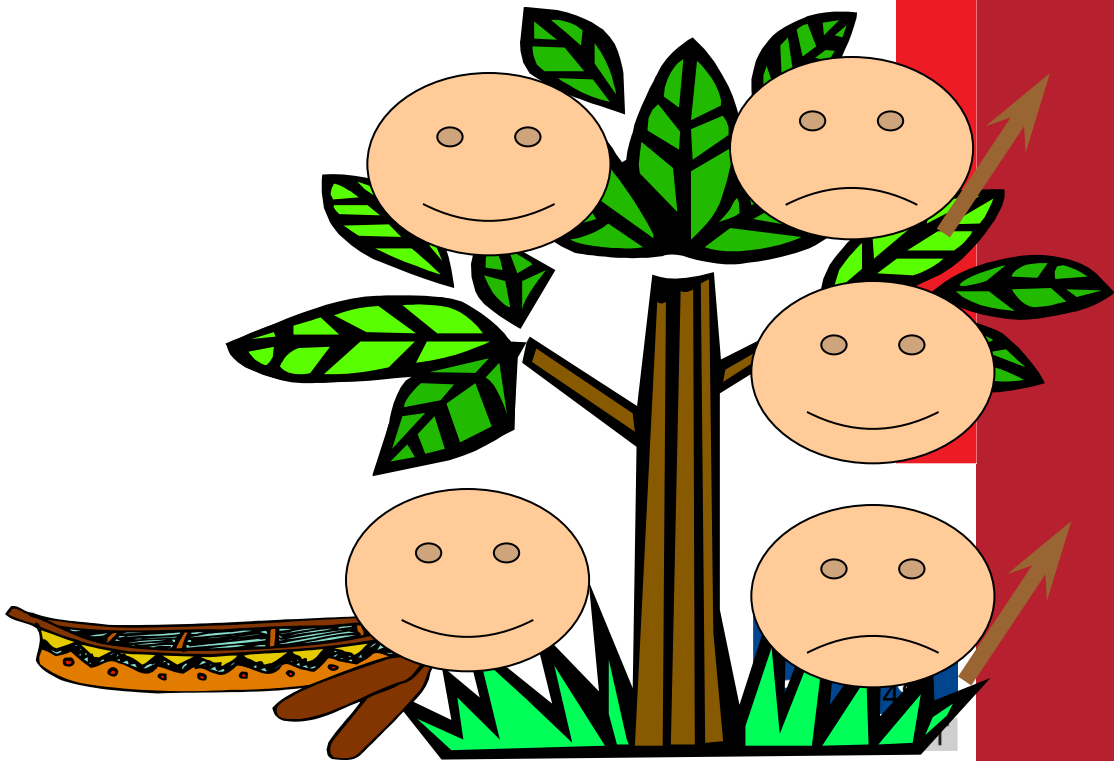




Missionaries and Cannibals

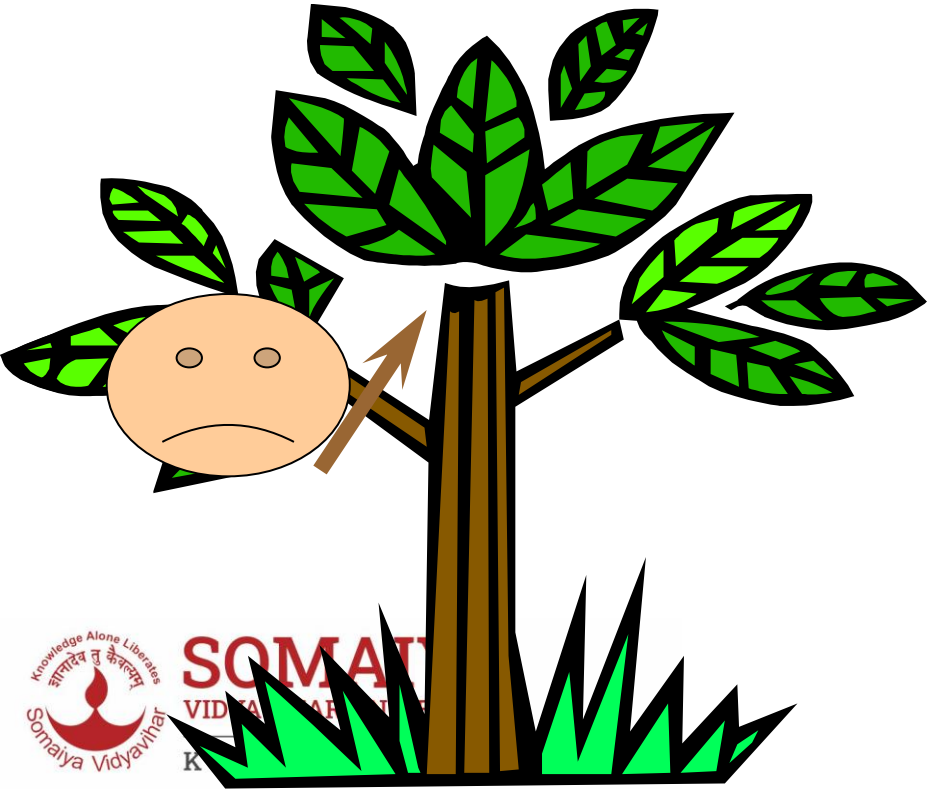
(0,1,0)

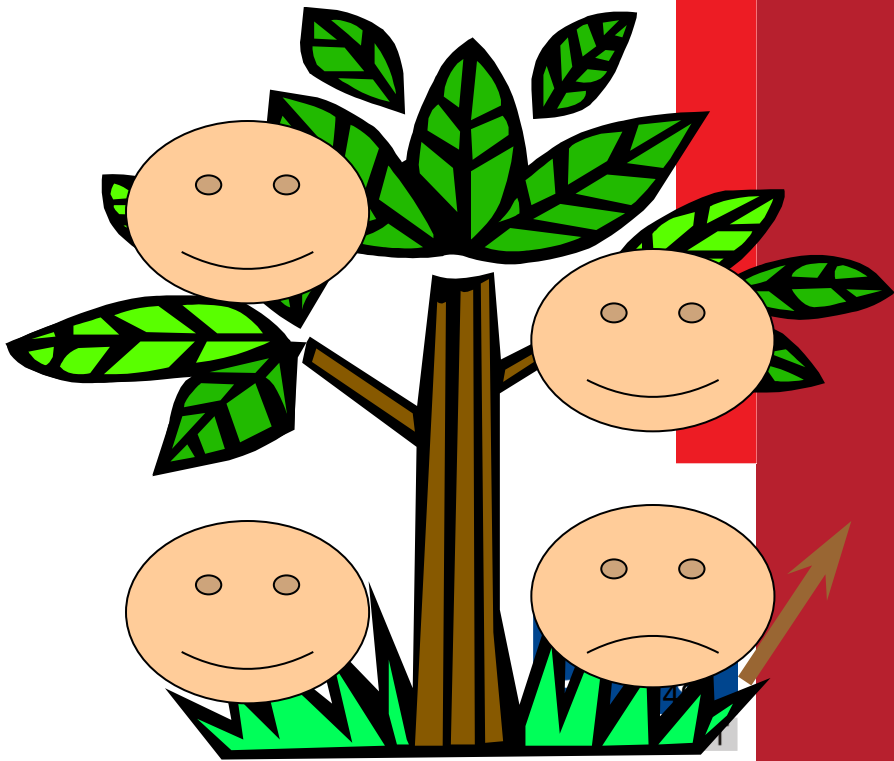




Missionaries and Cannibals

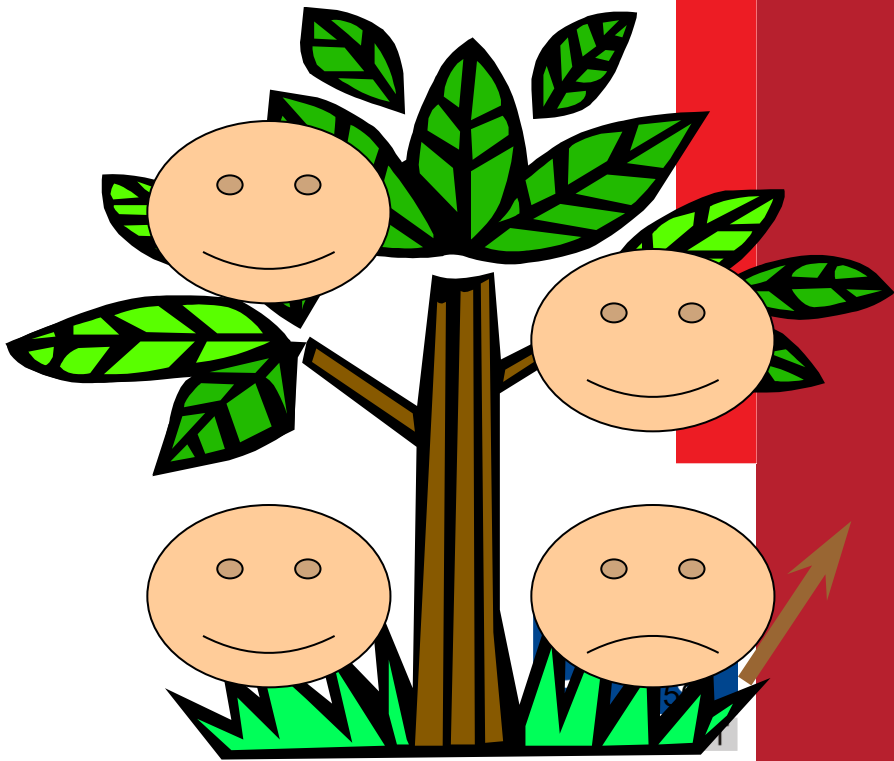
A cannibal returns





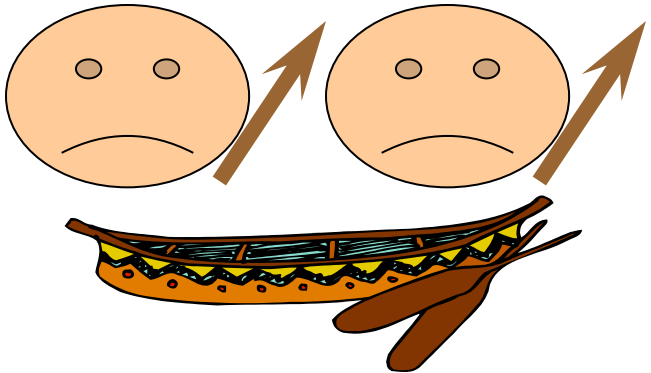
Missionaries and Cannibals

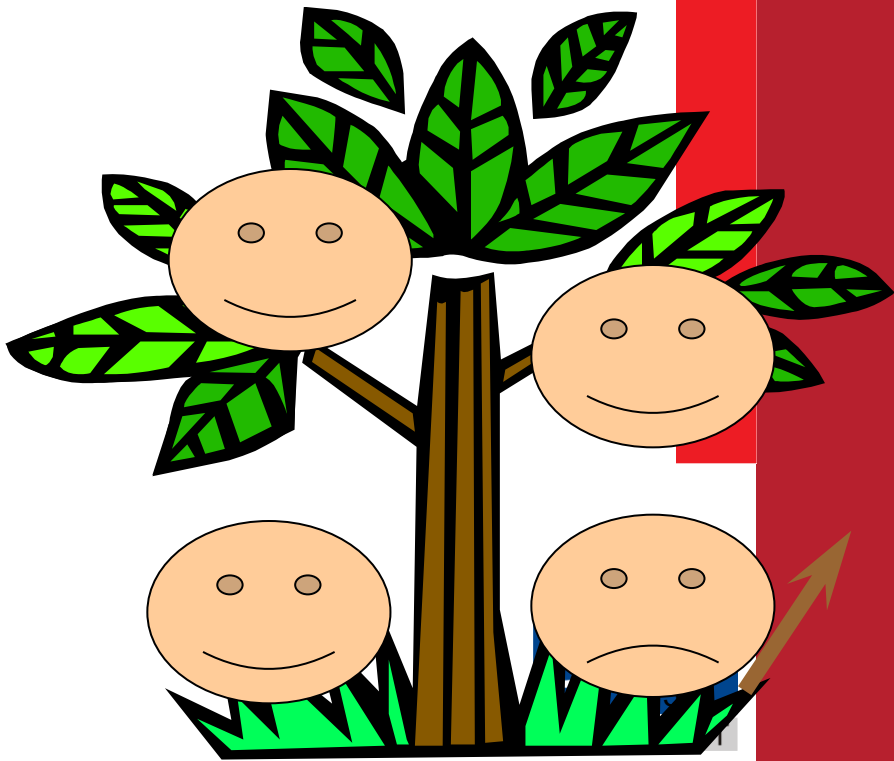
(0,2,1)



Missionaries and Cannibals

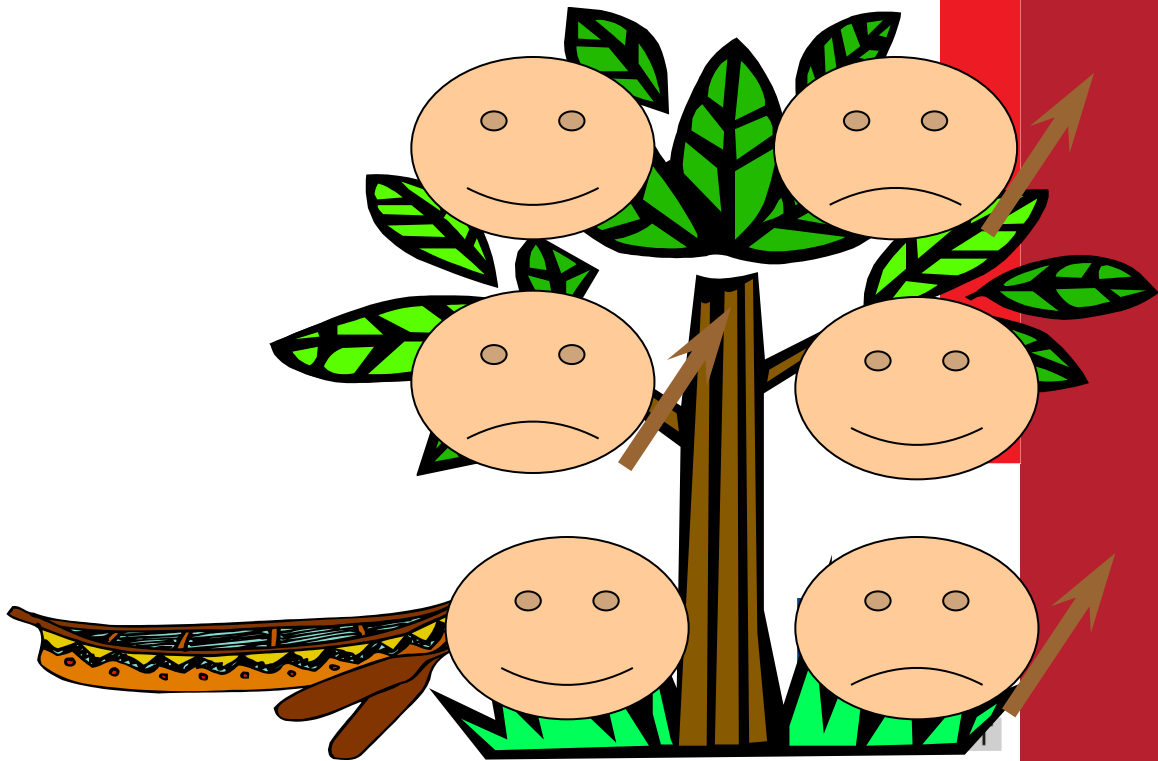
The last two cannibals cross





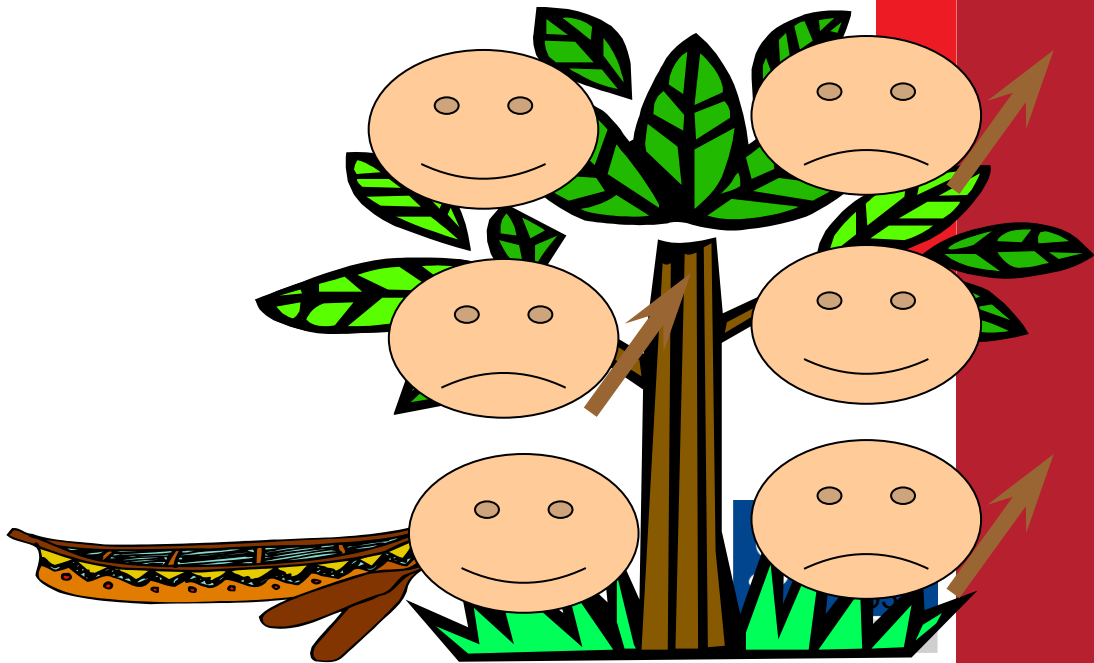
Missionaries and Cannibals

(0,0,0) : Goal State



Missionaries and Cannibals

Solution = the sequence of actions within the path : [(3,3,1)→ (2,2,0)→(3,2,1) →(3,0,0) →(3,1,1) →(1,1,0) →(2,2,1) →(0,2,0) →(0,3,1) →(0,1,0) → (0,2,1) →(0,0,0)]; Cost = 11 crossings



SOMAIYA
VIDYAVIHAR UNIVERSITY
K J Somaiya